



U N I V E R S I D A D
COMPLUTENSE
M A D R I D

**MÁSTER EN DATA SCIENCE, BIG DATA E INTELIGENCIA
ARTIFICIAL**

2025 - 2026

TRABAJO DE FIN DE MÁSTER

**“SISTEMA DE VISIÓN POR COMPUTADOR PARA
LA DETECCIÓN Y CLASIFICACIÓN DE
SEMÁFOROS PEATONALES COMO
HERRAMIENTA DE ASISTENCIA PARA
PERSONAS CON DISCAPACIDAD VISUAL”**

AUTOR: JAVIER GALDEANO VALENZUELA

MADRID, 17 DE SEPTIEMBRE DE 2026



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**.

RESUMEN

Muchas de las **acciones que realizamos día a día** y que no nos llaman especialmente la atención, **como por ejemplo cruzar una calle con uno o varios semáforos**, pueden convertirse en un **verdadero desafío para personas que padecen de una discapacidad visual**. Este trabajo de fin de máster busca aplicar varias de las técnicas de **Deep Learning aprendidas** durante dicho máster para **desarrollar un sistema de computer vision** que sea capaz de **detectar semáforos peatonales y clasificar su estado** así como estudiar su utilidad como herramienta de asistencia para la movilidad autónoma.

El **conjunto de datos** utilizado ha sido **construido desde cero** y está **formado por más de 180 vídeos** grabados **desde el punto de vista de un peatón** a lo largo de **varias localizaciones de Madrid**. Dichos vídeos fueron realizados **entre los meses de julio y agosto** e incluyen tomas **tanto diurnas como nocturnas**.

En cuanto a los **modelos utilizados**, se ha optado por la familia YOLO26 para la **detección** de los semáforos y una CNN propia para su **clasificación**; ambos **han sido evaluados por separado** tanto en **condiciones diurnas** como **nocturnas**, aunque **una parte del proyecto incluye una demo de Streamlit** donde se muestra el **funcionamiento conjunto**.

Con todo lo anterior, **este trabajo busca explorar qué potencial tiene la inteligencia artificial para ayudar a las personas con discapacidad visual** en la movilidad urbana y **abrir la puerta hacia la posibilidad de acercar dichas tecnologías a quien realmente las necesite**.

Palabras clave: computer vision; Deep Learning; YOLO; semáforos peatonales; discapacidad visual; clasificación de imágenes; CNN; movilidad; asistencia visual; redes neuronales convolucionales; accesibilidad; procesamiento de imágenes; detección de semáforos; inteligencia artificial; *explainable AI*; detección de objetos pequeños;

AGRADECIMIENTOS

A mi madre, por haberlo sacrificado siempre todo para que nunca me haya faltado nada.

A mi hermano, por renacer de entre el fuego y enfrentar la adversidad con valor.

A mi abuela, evidentemente, ¡qué sería de mí sin mi abuela!

A mis amigos: Yanni, Sheyla, Jhon y Daniel, por haberme ayudado a grabar vídeos para este proyecto y acompañarme con tanta cercanía a lo largo de todo el proceso. Os agradezco todo el apoyo que me habéis dado y que forméis parte de mi vida.

A mí, por haber tenido tantas oportunidades de rendirme y rechazarlas todas.

Y, por supuesto:

A mi pareja, Natalia, por cambiar mi vida y acompañarme sin dudarle en todas y cada una de mis locuras, por escucharme aun cuando ni yo mismo sé lo que digo, por apoyarme en cada idea, en cada proyecto y en cada sueño que se me ocurre. Por creer en mí de manera incondicional. Por su paciencia, por su amor, por elegir compartir cada día de su vida conmigo y por hacer que todo lo que hago y todo lo que soy merezca algo más la pena. Gracias por hacerme un mejor hombre y una mejor persona, eres la luz en este laberinto que llamamos vida; te amo.

PD: Algún día conseguiré convencerte para que me dejes enseñarte Python, lo sé.

ÍNDICE

I. INTRODUCCIÓN.....	6
I. I. CONTEXTO Y MOTIVACIÓN.....	7
I. II. OBJETIVO Y ALCANCE DEL PROYECTO.....	8
II. ESTADO DEL ARTE.....	9
III. METODOLOGÍA.....	10
III. I. ARQUITECTURA DEL SISTEMA.....	10
III. II. GENERACIÓN DE LOS DATOS.....	11
III. II. I. ADQUISICIÓN Y PROCESAMIENTO DE LOS VÍDEOS.....	11
III. II. II. ETIQUETADO DE IMÁGENES.....	12
III. II. III. GENERACIÓN DE LOS CONJUNTOS DE DATOS.....	13
III. III. DESARROLLO Y ENTRENAMIENTO DEL DETECTOR.....	14
III.IV. DESARROLLO Y ENTRENAMIENTO DE LA CNN.....	16
III. V. ORGANIZACIÓN Y REPRODUCIBILIDAD.....	18
IV. RESULTADOS.....	19
IV.I. RESULTADOS DEL DETECTOR YOLO.....	19
IV. I. I. COMPARATIVA ENTRE DATASETS.....	19
IV. I. II. COMPARATIVA ENTRE MODELOS.....	20
IV.II. RESULTADOS DEL CLASIFICADOR (CNN).....	21
IV. II. I. RESULTADOS EN VALIDACIÓN.....	21
IV. II. II. RESULTADOS EN TEST.....	23
IV.III. INTEGRACIÓN DE LOS MODELOS.....	24
V. CONCLUSIONES Y TRABAJO FUTURO.....	25
VI. REFERENCIAS.....	27
VII. ANEXOS.....	29
VIII.I. FIGURAS.....	29
VIII.II. TABLAS.....	45
VIII.III. DECLARACIÓN DE ORIGINALIDAD.....	50
VIII.IV. USO RESPONSABLE DE IA GENERATIVA.....	51
VIII.V. DISPONIBILIDAD DEL REPOSITORIO.....	52

ÍNDICE DE FIGURAS

Figura 1. Arquitectura general del sistema.....	29
Figura 2. Comparación entre la imagen original y la detección realizada por YOLO26.....	29
Figura 3. Etiquetado de semáforo peatonal utilizando Label Studio.....	30
Figura 4. Etiquetado de semáforo peatonal nocturno utilizando Label Studio.....	30
Figura 5. Semáforo nocturno no etiquetable.....	30
Figura 6. Contenido del fichero split_v1.json.....	31
Figura 7. Proceso de fine tuning de los modelos YOLO.....	31
Figura 8. Arquitectura de la CNN para la clasificación del estado de los semáforos.....	32
Figura 9. Data Augmentation en el entrenamiento de la CNN.....	32
Figura 10. Callbacks de la CNN.....	33
Figura 11. Resultados sobre test de YOLO26n según dataset utilizado, resolución y condiciones de luz. 33	
Figura 12. Resultados sobre test de YOLO26s según dataset utilizado, resolución y condiciones de luz. 34	
Figura 13. Bounding boxes e IoU según dataset para YOLOn y YOLOs 1088 píxeles y threshold 0.1.....	34
Figura 14. Resultados sobre test según modelo YOLO26 utilizado, resolución y condiciones de luz en dataset_v2.....	35
Figura 15. Evolución de métricas según resolución y modelo para dataset_v2.....	35
Figura 16. Relación entre el rendimiento diurno y nocturno de YOLO26 Nano y Small según resolución.....	36
Figura 17. ROC AUC de las CNN. Izquierda: dataset_v1; derecha: dataset_v2.....	36
Figura 18. Resultados CNN según versión del dataset utilizando Youden Threshold.....	37
Figura 19. Evolución de métricas en CNN según versión del dataset y threshold.....	37
Figura 20. False Positives and False Negatives en CNN según versión del dataset y threshold.....	38
Figura 21. Distribución de P(Green) según clase y umbrales Youden para cada dataset.....	38
Figura 22. Resultados CNN según versión del dataset utilizando Youden Threshold.....	39
Figura 23. Evolución de métricas en CNN según versión del dataset y threshold sobre test.....	39
Figura 24. Evolución de falsos negativos y falsos positivos según threshold sobre test.....	40
Figura 25. Falsos positivos y falsos negativos y su valor P(Green) para ambas versiones de CNN.....	40
Figura 26. Distribución de P(Green) según clase y umbrales Youden para test.....	41
Figura 27. Errores en test para la CNN dataset_v2.....	41
Figura 28. Errores en test para la CNN dataset_v2 y Grad-CAM++.....	42
Figura 29. Errores en test para la CNN dataset_v2 y Grad-CAM++.....	42
Figura 30. Interfaz de prototipo Streamlit.....	43
Figura 31. Semáforo nocturno apagado clasificado como Red.....	44
Figura 32. Semáforo nocturno correctamente clasificado como Green.....	44
Figura 33. Detección y clasificación de un cruce regulado por múltiples semáforos.....	44

ÍNDICE DE TABLAS

Tabla 1. Resumen de los conjuntos de datos obtenidos.....	45
Tabla 2. mAP50-95 y número de parámetros (en millones) según tipo de modelo YOLO. 640 x 640 píxeles de referencia.....	45
Tabla 3. Estadísticas básicas de las bounding boxes según dataset.....	45
Tabla 4. Hiperparámetros básicos del entrenamiento YOLO.....	46
Tabla 5. Resultados del detector YOLO según dataset, modelo y resolución en test diurno.....	46
Tabla 6. Resultados del detector YOLO según dataset, modelo y resolución en test nocturno.....	46
Tabla 7. mAP@50:95 en test diurno según modelo y resolución y diferencia en porcentaje.....	47
Tabla 8. mAP@50:95 en test nocturno según modelo y resolución y diferencia en porcentaje.....	47
Tabla 9. Número de imágenes y distribución por clase de los datasets empleados en la CNN.....	47
Tabla 10. Resultados del clasificador utilizando Youden según versión del dataset.....	48
Tabla 11. Comparativa de resultados del clasificador para dataset_v2 según threshold.....	48
Tabla 12. Resultados de los clasificadores utilizando el conjunto de test.....	49

I. INTRODUCCIÓN

I. I. CONTEXTO Y MOTIVACIÓN

La **discapacidad visual** es una realidad muy presente y que **afecta a un considerable número de personas en todo el mundo, impactando negativamente en su calidad de vida: sobre todo, en su autonomía**. Según datos de la Organización Mundial de la Salud (OMS, 2026), “al menos **2200 millones de personas presentan deterioro de la visión, cercana o lejana**”. Estimaciones realizadas por el *Global Burden of Disease Study* sitúan que, para el año 2020, en **43 millones el número de personas que sufrían de ceguera**. A estas se deben sumar otros **295 millones con discapacidad moderada o grave** (Bourne et al., 2020).

Los autores también **señalan los riesgos para la discapacidad visual de un panorama demográfico en donde la población envejece progresivamente**. Prieto González (2025) comenta que “**en 2050 la mitad de la población mundial será miope**” y que, por tanto, “**cualquier servicio diseñado hoy debe contemplar este escenario**”.

En **ámbito nacional** los datos proporcionados por la *International Agency for the Prevention of Blindness* (IAPB, s. f.), sitúan en **aproximadamente 2,84 millones el número de personas ciegas o que sufren un deterioro de carácter moderado o grave en la visión**. Además, se observa que **la ceguera no afecta a ambos géneros por igual, siendo la prevalencia en mujeres el doble que en hombres**, con 0,162 millones frente a 0,08.

La **Organización Nacional de Ciegos Españoles** (ONCE, s. f.-a), una de las instituciones españolas de referencia en materia de discapacidad visual, **compartió, en el año 2025, unas cifras totales de 72.443 personas afiliadas**, dentro de las que solo forman parte aquellas personas que cumplen con una serie de criterios, considerablemente estrictos, establecidos por dicha organización (ONCE, s. f.-b). Aun así, sirve como un **indicador de la magnitud del problema dentro del territorio nacional**.

Además de las cifras mencionadas, es necesario comprender cómo **influye la discapacidad visual en la autonomía diaria** de las personas afectadas. Una de las principales funciones que se ve comprometida es el **desplazamiento autónomo**, recogido por la ONCE (s. f.-c) como “**algo esencial para poder realizar actividades de la vida cotidiana tanto en el hogar como fuera de él**”. La asociación además hace hincapié en que **la discapacidad visual dificulta el acceso a fuentes de información** que resultan **imprescindibles para tener una movilidad adecuada**, como por ejemplo **los semáforos**,

lo que en muchos casos acaba **obligando** a la persona afectada, lamentablemente, a **depende**r de los demás.

Armengol (2026), **periodista y divulgador en redes sociales con discapacidad visual**, muestra en sus vídeos varios de los desafíos a los que debe enfrentarse en su día a día. Uno de sus vídeos, utilizado como referencia para este trabajo, describe la **complejidad de cruzar un paso de peatones** cuando está señalizado por **varios semáforos**, donde además coincide que **cada uno marca un estado distinto**. Es precisamente como respuesta a esta clase de situaciones que nace este **Trabajo de Fin de Máster**.

I. II. OBJETIVO Y ALCANCE DEL PROYECTO

El objetivo del presente TFM consiste en **desarrollar un sistema completo de computer vision utilizando Deep Learning para la detección de semáforos peatonales y la posterior clasificación de su estado**. De esta manera se busca que pueda servir como una **herramienta para asistir a personas con discapacidad visual** y que puedan **ganar autonomía en los desplazamientos de carácter urbano**.

Para ello, **se deberá:**

1. **Construir un dataset propio** compuesto de **vídeos grabados desde la perspectiva de un peatón** que intenta atravesar un cruce señalizado por semáforos, **procesar dichos vídeos para extraer los frames** relevantes y posteriormente **realizar las anotaciones necesarias de cara a entrenar los diferentes modelos**.
2. **Desarrollar varios modelos de detección** basado en *object detection*, utilizando la familia YOLO 26 en dos de sus **diferentes tamaños (Nano y Small)** y para **distintas resoluciones** de imagen.
3. **Diseñar una red neuronal convolucional para la clasificación** del estado de los semáforos detectados. Dichos estados serán: **Red y Green**.
4. **Evaluar los modelos** utilizando para ello un **conjunto de datos completamente independiente** con vídeos/imágenes **tanto de día como de noche**.
5. **Integrar los diferentes modelos dentro de un prototipo** que permita cargar videos y visualizar el funcionamiento **end-to-end del proyecto**.
6. **Garantizar la máxima reproducibilidad** del proyecto, mediante su subida a un **repositorio de GitHub**, el uso de **semillas** y la **documentación de los splits** utilizados para train/test en cada prueba así como de todo lo necesario para lograr dicha reproducibilidad.

El alcance del proyecto queda limitado a lo anteriormente descrito. No se busca **desarrollar ninguna aplicación funcional**, sino **estudiar su viabilidad** de cara a futuras aplicaciones. El **prototipo desarrollado cuenta únicamente con la finalidad de**

demostrar el funcionamiento conjunto de los modelos y servir como un acercamiento al posible desarrollo de aplicaciones de asistencia visual mediante Deep Learning.

II. ESTADO DEL ARTE

La detección y clasificación automática de los semáforos ha sido, desde hace años, un **asunto con gran atractivo** dentro de la **comunidad científica** y más aún dentro del campo de la **visión por computador**. Entre los estudios utilizados como referencia para el presente proyecto encontramos el trabajo de **Behrendt et al. (2017)**, más enfocado a la **conducción autónoma** donde se propone una arquitectura similar a la empleada en el presente trabajo: un **detector mediante YOLO**, un **clasificador CNN** con recortes **64 x 64 píxeles x 3 canales** y otra CNN para descartar falsos positivos y falsos negativos. En el mencionado estudio, además, **se destaca la dificultad de detectar semáforos dado su reducido tamaño**.

Más concretamente, en materia de discapacidad visual, **Cheng et al. (2018)** desarrollaron un sistema *in real-time* para la detección de semáforos peatonales utilizando unas **gafas con sensor Intel RealSense R200**, un par de auriculares y un ordenador portátil. Para ello, se utilizaron **técnicas de segmentación por color y una clasificación mediante máquinas de vector soporte junto con análisis espacio-temporales**.

Posteriormente, con la evolución del **Deep Learning**, **Yu et al. (2019)** proponen **LYTNet, una única red neuronal convolucional** diseñada con el objetivo de ayudar a personas con discapacidad visual a la hora de cruzar la calle, **clasificando el estado del semáforo y analizando la dirección del paso de cebra**. Para ello, desarrollaron un **dataset propio de 5059 imágenes** y, como resultado, lograron una **arquitectura ligera y capaz de ejecutarse en un dispositivo móvil** mediante Swift.

El-Taher et al. (2021), en su artículo «**A Systematic Review of Urban Navigation Systems for Visually Impaired People**» realizan una revisión de los sistemas de navegación urbanos enfocados a personas con discapacidad visual, donde **sitúan la detección y la clasificación de los semáforos peatonales como una de las tareas más relevantes** para lograr una correcta movilidad urbana. Los propios autores señalan que **“reconocer los semáforos peatonales es una tarea significativa para que las personas con discapacidad visual puedan saber cuándo cruzar una calle”**, además de **identificar varias de las dificultades** asociadas, como los **diferentes tamaños** o formas de los semáforos, la **inestabilidad de los movimientos de la cámara (shaking)** o los cambios en la **iluminación**. También distinguen entre dos **tipos de enfoque**: aquellos que utilizan un

único modelo y aquellos que separan el proceso en **dos o más etapas**, perteneciendo este trabajo al segundo grupo.

Estudios más recientes, como el de **Ismail y Mousa (2025)** ya **desarrollan soluciones** basadas en modelos **optimizados para smartphones** donde un mismo modelo **YOLOv11 Nano detecta y clasifica los semáforos peatonales** en función de si su estado es rojo o verde.

Por último, **Jocher et al. (2026)** presentan los modelos **YOLO26**, una generación que implementa **tecnología STAL (Small-Target-Aware Label Assignment)** que permite un **mejor aprendizaje y detección de objetos pequeños**, lo cual resulta de especial relevancia para la naturaleza del **presente proyecto**, donde a diferencia de Ismail y Mousa (2025) **se utiliza un modelo YOLO 26 exclusivamente para la detección de los semáforos, dejando su clasificación a una CNN especializada.**

III. METODOLOGÍA

III. I. ARQUITECTURA DEL SISTEMA

La **arquitectura** del presente trabajo **ha sido pensada de manera modular**, compuesta por **dos etapas independientes**: una primera encargada de **la detección de los semáforos** peatonales y una segunda cuyo objetivo es **la clasificación de su estado**. De esta manera, se **facilita el entrenamiento y la evaluación** de cada etapa **así como la detección y corrección de errores** que puedan originarse en alguna de las partes.

Tal y como se observa en la [Figura 1.](#), el sistema está pensado para **comenzar con una imagen o vídeo** de entrada que será **procesado por un modelo basado en YOLO26**, entrenado **exclusivamente para la detección de semáforos peatonales**, cuya etiqueta es **pedestrian_traffic_light** siendo esta completamente independiente del estado del semáforo —siendo válida incluso para semáforos apagados—. Dicho modelo, tal y como se aprecia en la [Figura 2.](#), **obtiene las bounding boxes** que corresponden a la posición de los semáforos detectados **y extrae los recortes** de la imagen, que posteriormente **serán reescalados a 128 x 128 píxeles** para ser utilizados **como entrada de la CNN**, encargada de **clasificar el estado** de dicho semáforo. Al tratarse este último paso de una clasificación binaria, **se obtiene la predicción mediante $P(\text{Green})$ y un threshold definido.**

El uso de un sistema en dos etapas **aporta además una mayor flexibilidad** al permitir **combinar diferentes configuraciones** de detector y clasificador **en función de las necesidades** del sistema donde se ejecute. Esto **facilita**, por ejemplo, **evaluar la**

performance con distintas configuraciones en dispositivos de recursos limitados como un teléfono móvil.

III. II. GENERACIÓN DE LOS DATOS

III. II. I. ADQUISICIÓN Y PROCESAMIENTO DE LOS VÍDEOS

Para el desarrollo del trabajo ha sido necesario crear un **conjunto de datos propio**, ya que **los estudios mencionados anteriormente utilizaron datos obtenidos en otros países**, donde tanto la **forma de los semáforos** como las **características de los entornos urbanos** pueden ser **considerablemente diferentes de los españoles**. Por ello, con el objetivo de disponer de **datos más representativos**, se han recopilado vídeos de **semáforos peatonales en diferentes ubicaciones de Madrid**.

Dichas grabaciones **se realizaron por varias personas con diferentes modelos de teléfono móvil y desde un punto de vista peatonal**. Para **aumentar la diversidad**, estas grabaciones se realizaron con **diferentes condiciones lumínicas, distancias y ángulos así como con diferentes situaciones urbanas —con presencia de vehículos, reflejos, carteles luminosos y peatones**. De esta forma, se pretendía **evitar un dataset controlado** y se apostó por uno **más representativo de la realidad**.

Una vez recopilados, **estos vídeos fueron procesados** mediante *scripts* desarrollados especialmente para el proyecto. De esta forma **todos fueron normalizados a un formato .mp4** para posteriormente **automatizar la extracción de los fotogramas** que serían utilizados en el proceso de etiquetado. **Los videos fueron grabados a 60 fotogramas por segundo y en su inmensa mayoría siguiendo el estándar 1920 x 1080 píxeles**, y para mantener una muestra representativa de cada segundo de vídeo se **extrajeran 3 fotogramas**.

Como resultado, y **siguiendo los datos aportados en la [Tabla 1](#)**, el conjunto de datos utilizado contó con una **primera versión** formada por **32 vídeos**, de los que se obtuvieron **2.197 fotogramas y 3.392 semáforos anotados**. Con el objetivo de **aumentar la estabilidad de los modelos y la variedad muestral**, se creó una **segunda versión** formada por **99 vídeos, 4.090 fotogramas y 6.332 semáforos anotados**.

El hecho de que **el número de semáforos anotados sea superior al número de fotogramas** se debe a que **determinadas imágenes contienen más de un semáforo peatonal simultáneamente**. Del mismo modo, **el número final de fotogramas obtenidos depende de la cantidad de vídeos, pero más de la duración de estos**.

Por último, **se recopilaron dos conjuntos completamente independientes** destinados a la evaluación final de los modelos: **un dataset diurno, compuesto por 39 vídeos, 718 fotogramas y 816 semáforos anotados; y un dataset nocturno formado por 30 vídeos, 1.119 fotogramas y 1.119 objetos**.

La **normalización** de los vídeos y la **extracción de los fotogramas** se automatizaron mediante los scripts `video_converter.py` y `extract_frames.py`, disponibles en el repositorio del proyecto.

III. II. II. ETIQUETADO DE IMÁGENES

Una vez extraídos los diferentes fotogramas, se procedió a su **etiquetado manual mediante Label Studio**. Dicha herramienta está **diseñada para facilitar la creación de datasets** para distintos **modelos de aprendizaje automático y profundo**, entre ellos los YOLO (**You Only Look Once**). Y para favorecer un **dataset coherente y un entrenamiento estable**, se establecieron **criterios homogéneos** de etiquetado de forma que se aplicaron las **mismas reglas a todos los semáforos** peatonales.

Dicho criterio consistió en **etiquetar únicamente el panel y la caja** del semáforo peatonal como unidad, **de la forma más ceñida posible y evitando incluir postes u otros elementos del entorno** dentro de la *bounding box*, siempre que el **semáforo** fuese **identificable por un ser humano sin** necesidad de hacer **zoom o inferir su silueta**. Para semáforos cuyo ángulo no permitiese ajustarlo correctamente, como el reflejado en la [Figura 3.](#), se buscó **encajar sus extremos**, de forma que **se incluyese la menor cantidad de información irrelevante**.

No obstante, **en imágenes nocturnas, fue necesario flexibilizar dicho criterio** dado que condiciones de **baja luz** o la existencia de **reflejos o destellos**, bien producidos **por los propios semáforos o por otros elementos del entorno**, podían **dificultar** la labor de **delimitar correctamente los bordes** del semáforo. En consecuencia, como se observa en la [Figura 4.](#), siempre que el semáforo siguiese siendo identificable, se **permitió aplicar una bounding box aproximada** al inicio y fin de su contorno **siempre que esta resultase coherente**. En caso contrario, tal y como puede comprobarse en la [Figura 5.](#), **no se etiqueta**.

Para el etiquetado de las imágenes **se utilizó una única clase** denominada `pedestrian_traffic_light`, **con independencia del estado** del semáforo. Adicionalmente, **cada semáforo fue asociado a uno de tres posibles estados: Red, Green y Off**. Esta última fue utilizada cuando el semáforo se apaga en un cambio de verde a rojo o si el semáforo se encontraba averiado.

Además de lo anterior, también **se utilizaron imágenes en las que no aparecía ningún semáforo, denominadas como backgrounds**. Estas fueron conservadas de forma que **el modelo pudiese aprender que no siempre existe un semáforo** en la imagen.

Todo el proceso de etiquetado fue realizado de forma manual, sin utilizar modelos pre entrenados ni técnicas Human in the Loop. Una vez realizado el proceso de etiquetado, **las anotaciones fueron exportadas a formato JSON desde Label Studio** y se procedió a la generación de los conjuntos de datos.

III. II. III. GENERACIÓN DE LOS CONJUNTOS DE DATOS

Una vez **descargado el JSON con todas las anotaciones** del dataset, estas fueron **procesadas mediante scripts desarrollados** en específico para dicha tarea, transformando las **coordenadas al formato requerido por YOLO**.

Para generar físicamente el dataset **se realizó la división por vídeo completo y no por fotograma**. Esta decisión se tomó debido a que **los fotogramas dentro de un mismo vídeo presentan una gran similitud entre ellos**. Y si algunos de ellos perteneciesen al conjunto de entrenamiento y otros al de validación, **podría producirse data leakage**.

Además, **se establecieron dos objetivos adicionales** para la división de los datos: mantener **un ratio aproximado del 70% para entrenamiento y del 30% para validación**, y conservar **una distribución lo más equilibrada posible entre los estados Green y Red**. Sin embargo, y **dado que cada vídeo presentaba una duración diferente, y que algunos contenían principalmente semáforos rojos y otros verdes**, fue necesario establecer una **estrategia de división** que garantizase el cumplimiento de ambos objetivos.

De esta manera, **el script `generate_yolo_dataset.py` genera 10.000 divisiones aleatorias y selecciona aquella que cumpla de la mejor manera ambos objetivos**. Estas divisiones **se generan utilizando una semilla fija y el split generado se almacena para poder ser reutilizado**.

Así, **el proceso de generación de los datasets está diseñado para trabajar con diferentes versiones de los datos**. Cada versión queda asociada a unas **anotaciones exportadas** desde Label Studio **así como a un archivo de split** específico. Cuando se genera el **dataset** para una **versión donde ya se generó un split**, **el archivo se recupera y se reutiliza** de manera que se mantiene la misma división de los videos en train y validación que se utilizó originalmente. En caso contrario, **si no existe ningún split** porque la versión de los datos es nueva, **el sistema carga las anotaciones, exportadas desde label studio** que corresponde y ejecuta el proceso descrito, **guardando el resultado en un archivo de split**.

De esta forma, **el versionado del dataset no solo se limita a las anotaciones, sino también a las propias particiones** utilizadas en cada entrenamiento.

Para representar lo anterior de forma visual, **la [Figura 6](#) muestra el contenido del archivo `split_v1.json`**, en el que se **almacenan los identificadores de los vídeos** —000XXX, siendo los tres últimos dígitos el número de vídeo—, **asignados a cada conjunto: entrenamiento y validación**. **Su reutilización permite reconstruir el dataset manteniendo exactamente la misma partición utilizada en su momento**, por ejemplo, **para realizar pruebas adicionales** en modelos entrenados con versiones específicas de los conjuntos de datos **o para reproducir los entrenamientos**.

III. III. DESARROLLO Y ENTRENAMIENTO DEL DETECTOR

Para la **primera etapa** del sistema se seleccionó la familia de **modelos YOLO**, debido a su **facilidad de uso** y por ofrecer **un amplio abanico de modelos** ([Tabla 2.](#)), características de especial relevancia teniendo en cuenta que una **posible evolución futura** del proyecto podría ser su implementación **en dispositivos móviles**.

Para ser más precisos, **se utilizó la familia YOLO26, la versión más reciente desarrollada por Ultralytics a la fecha del proyecto**. Este nuevo modelo **incluye** como principal novedad **la estrategia STAL (Small-Target-Aware Label Assignment)**, diseñada para **facilitar el aprendizaje** proveniente de **objetos pequeños** y su posterior detección (Ultralytics, 2026a). Esta característica es **especialmente importante** para este proyecto dado que **los semáforos peatonales ocupan**, en determinados casos, **un porcentaje muy pequeño de la imagen** y, al redimensionar esta para el entrenamiento, dichos semáforos **pueden quedar reducidos a un número muy pequeño de píxeles**.

Para respaldar la afirmación anterior, se adjunta la [Tabla 3.](#), donde se muestran **las dimensiones relativas**, en porcentaje sobre el total de la imagen, **de las bounding boxes anotadas**. Como puede observarse, en los **datasets de entrenamiento** la **anchura y altura medianas de los semáforos se sitúan en torno al 2.7%**, aunque **existen casos con dimensiones inferiores al 1%**. Estos valores reflejan el hecho de que **una parte importante de los semáforos ocupa una región muy pequeña** dentro de las imágenes, respaldando la elección de YOLO26 como detector.

Con el objetivo de **evaluar el equilibrio entre el coste computacional y métricas** como la precisión, *recall* o el *mAP*, se evaluaron **dos variantes principales de YOLO: YOLOn –Nano–**, su versión más pequeña con **2.4 millones de parámetros**, y **YOLOs –Small–**, una versión ligeramente mayor con aproximadamente **9.5 millones de parámetros** —utilizando como referencia una imagen de 640 x 640—, (Ultralytics, 2026a). Dado el **incremento en parámetros que supone el siguiente escalón** de la familia así como el coste computacional asociado, **no se utilizaron variantes de mayor tamaño**. Además, **se quiso mantener** el experimento, como se ha mencionado a lo largo de este informe, **dentro de configuraciones compatibles con dispositivos de bajos recursos**.

Dentro de cada configuración se evaluaron cuatro resoluciones de entrada: **640, 800, 960 y 1088 píxeles**, dando un total de **ocho combinaciones posibles**. Esta **decisión está relacionada con el reducido tamaño de los semáforos**. Aumentar la resolución de la imagen permite **mantener más información** y puede **facilitar la detección y delimitación de los objetos pequeños**. Sin embargo, también **incrementa el coste computacional**.

El detector de semáforos fue entrenado utilizando **una única clase denominada pedestrian_traffic_light** independientemente del estado del semáforo. De esta

manera, YOLO únicamente se encarga de localizar los semáforos, mientras que la clasificación depende de la CNN, en la segunda etapa.

Estos modelos **no fueron entrenados de cero, sino que se utilizó *transfer learning* y *fine-tuning***. Esto se debe a que los YOLO26 utilizados fueron **preentrenados** y ajustados **sobre Object365v1 y COCO** (Ultralytics, 2026b). Dichos conjuntos **no contienen la clase necesaria** para detectar semáforos peatonales, **por lo que fue necesario entrenarlos con el dataset desarrollado** para este proyecto.

Para la **primera versión** del dataset **se utilizaron los pesos oficiales** de YOLO26 tanto para la versión *Nano* como la *Small*. Posteriormente **durante el segundo** entrenamiento, **cada configuración se inicializó utilizando el mejor checkpoint de cada arquitectura en específico sobre el dataset_v1**. Es decir, **para entrenar la segunda versión de YOLO26n a 640**, se partió de los **mejores pesos** obtenidos en el **entrenamiento de la misma configuración** para la primera versión. Dicho flujo se muestra de manera más accesible y esquemática en la [Figura 7](#).

La configuración establecida para los entrenamientos está recogida en la [Tabla 4](#), y constó de un **máximo de 75 épocas** con un criterio de ***early stopping* con paciencia de 20**, de forma que dicho entrenamiento pudiese terminar antes en caso de no mejorar. **El optimizador se mantuvo en automático y el tamaño del batch fue, en general, de 8 imágenes**.

No obstante, **algunas configuraciones** correspondientes a la segunda versión del dataset **fueron probadas con un batch size de 16** con el objetivo de comprobar si se podía agilizar el entrenamiento. **Sin embargo, al aumentar la resolución de entrada comenzaron a aparecer limitaciones con la memoria disponible de la GPU (VRAM)**, por lo que **se volvió a utilizar el batch size original**.

Cada entrenamiento generó además un fichero *args.yaml* en el que quedaron **registrados los parámetros utilizados**. De esta forma, **cada una de las ocho configuraciones en ambas versiones cuenta con un registro independiente** que facilita tanto la **consulta como la reproducción** de los experimentos.

En dichos entrenamientos **también se aplicó *data augmentation* para mejorar la generalización del detector**. Más concretamente, se utilizaron **volteos horizontales, pequeñas rotaciones de hasta 3 grados y traslaciones**. No se incorporaron más modificaciones porque, si bien pueden ayudar a reducir el overfitting, **la perspectiva de la captura del video y la orientación de los semáforos son, por lo general, estables**. Por lo tanto, **se consideró que introducir más variaciones podía generar situaciones visuales poco realistas o representativas o dificultar la convergencia de los modelos**.

Para evaluar la *performance* de los modelos se utilizaron las métricas: **precisión** —qué porcentaje de las detecciones realizadas por el modelo corresponden efectivamente a un semáforo—, **recall** —qué porcentaje de los semáforos ha sido detectado satisfactoriamente—, **mAP@50 y mAP@50:95** —las cuales utilizan la

intersección entre el área de la caja predicha y la caja real con respecto al área de la unión de estas (*Intersection over Union*), comparándola con el threshold marcado—. **Estas últimas métricas cobran especial relevancia dado que la bounding box generada por el modelo YOLO será la utilizada por la CNN para la clasificación de la clase. Por tanto, interesa detectar correctamente un semáforo, pero también delimitarlo con suficiente precisión de manera que tenga la menor cantidad de información irrelevante.**

De hecho, el **criterio oficial de Ultralytics** para determinar el **mejor checkpoint** asigna todo el peso del resultado al **mAP@50:95** (Ultralytics, 2026d).

III.IV. DESARROLLO Y ENTRENAMIENTO DE LA CNN

Para la **segunda etapa** del sistema se desarrolló una **red neuronal basada en convoluciones** encargada de **clasificar el estado de los semáforos** previamente localizados.

Para generar los conjuntos de datos **se partió de las bounding boxes anotadas manualmente utilizando Label Studio**. A partir de dichas coordenadas, **se extrajeron los recortes de cada semáforo, evitando utilizar en el entrenamiento ninguna detección generada por YOLO**. De esta manera, tanto el **detector** como el **clasificador** son **entrenados a partir del mismo origen, reduciendo el riesgo de que errores de detección en la primera etapa perjudiquen el entrenamiento de la segunda**.

Los **recortes** fueron **redimensionados a 128 x 128 píxeles con tres canales de color**, siguiendo el estándar **RGB**, de forma que, con independencia del tamaño original del recorte, **la red recibe una imagen estandarizada**. Esta **resolución** se consideró **equilibrada** entre información conservada y coste computacional, aunque **se podría profundizar más** acerca de su influencia **en futuras líneas de investigación**. Antes de llegar a las capas convolucionales, **también se aplicó un reescalado, transformando la escala de valores de los píxeles de [0, 255] a [0, 1]**.

Durante el etiquetado de las imágenes se distinguieron tres estados para un semáforo: **Red, Green y Off**. Sin embargo, **el problema de clasificación ha sido planteado finalmente como un dilema binario**, utilizando las etiquetas: **Red = 0 y Green = 1**, y **reasignando los semáforos correspondientes al estado Off a la clase Red**. Dicha decisión busca **garantizar una postura conservadora en cuanto al riesgo** y establece que, **si no es posible interpretar de manera clara que el estado del semáforo permite cruzar la calle con seguridad, entonces no debería producirse dicho cruce o al menos no debería recomendarse**.

La **arquitectura utilizada por la CNN** se puede observar con mayor detenimiento en la [Figura 8](#), y **está compuesta por cuatro bloques convolucionales** donde el **número de filtros aumenta** en cada bloque, siendo la **progresión de 32, 64, 128 y 256**. Cada uno

de estos **bloques** está **compuesto a su vez de dos capas convolucionales 2D** donde se utilizan **kernels 3 x 3 y padding = same**. Asimismo, **después de cada convolución se ha aplicado Batch Normalization, una regularización L2 y una función de activación SiLU**. Por último, **al final de los tres primeros bloques se aplica un MaxPooling de 2 x 2**. De esta forma, a la par que **el número de filtros aumenta**, se va **reduciendo el tamaño de los feature maps**.

Tras el último bloque convolucional se aplica un Global Average Pooling de manera que se pueda **reducir notablemente el número de parámetros para la capa densa** y mantener una **arquitectura más reducida**. Posteriormente se aplica **un primer Dropout del 30%**, seguido de **una capa densa de 256 neuronas** —también con activación SiLU—, **un segundo dropout del 20%**, otra **capa densa de 64 neuronas** y finalmente **una capa de salida formada por una neurona** cuya función de activación es una **sigmoide**. Esta última fue **entrenada para determinar la probabilidad de que el estado del semáforo sea verde —P(Green)—**. Por tanto, valores cercanos a cero indican un estado Red.

Durante el entrenamiento también **se aplicaron técnicas de data augmentation** sobre las imágenes de entrada. Esto, **con el fin de aumentar la variedad muestral y mejorar la capacidad de generalización de la CNN**. Más concretamente, y como se puede comprobar en la [Figura 9](#), se utilizaron **volteos horizontales, rotaciones y traslaciones, zoom y variaciones de contraste**. Al igual que en el detector, **se evitó introducir transformaciones muy agresivas** que pudiesen plantear situaciones alejadas de la realidad.

Para el entrenamiento de la CNN **se utilizó BinaryCrossentropy** como función de pérdida —dado el carácter binario del problema— y **AdamW como optimizador** con un **learning rate inicial de 3e-4 y un weight decay de 1e-4**, lo que introduce un mecanismo de regularización sobre los pesos del modelo. Se configuró **un máximo de 75 épocas**, aunque de la misma manera que con el detector, **se fijaron ciertas estrategias de early stopping**.

En concreto, los **callbacks** quedan recogidos en la [Figura 10](#). Se utilizó **ReduceLROnPlateau**, que reduce el **learning rate** si no se observan mejoras en el **val_loss** durante tres épocas. En paralelo, un **EarlyStopping** permite detener el entrenamiento si el modelo no mejora durante 15 épocas, restaurando los pesos correspondientes al mejor resultado. Por último, **utilizando ModelCheckpoint se almacena únicamente el modelo con el mejor val_loss**.

Aquí, a diferencia con lo realizado en el detector, **todas las CNN fueron entrenadas de cero, con independencia de la versión del dataset utilizado**. Esta decisión se tomó **debido al menor coste computacional de estos entrenamientos** en comparación con el que se requirió para el detector.

Para **reducir el riesgo de overfitting** se utilizaron varios mecanismos. Además del **data augmentation**, se aplicó **BatchNormalization y regularización L2** a las capas

convolucionales, **dos capas de Dropout** en el bloque denso, el *weight decay* que proporciona AdamW así como los *callbacks* mencionados anteriormente.

Finalmente, **como punto de partida se fijó el threshold de 0.5** para determinar cuándo una predicción debería considerarse positiva. No obstante, **dicho valor no se consideró definitivo** y en apartados posteriores **se ahondará en dicho parámetro y en el consecuente comportamiento del clasificador.**

III. V. ORGANIZACIÓN Y REPRODUCIBILIDAD

El proyecto **se estructuró de forma modular**. Los vídeos originales, anotaciones y recortes generados **se almacenaron** dentro de */data*, mientras que todos los distintos scripts **se organizaron en sus respectivas carpetas según la función que realicen**: generación de datasets, entrenamiento, evaluación etc. A lo largo del desarrollo, **se trató de centralizar todas las funciones de carácter reutilizable dentro de */scripts/utils***, de manera que los principales flujos de trabajo quedasen limpios y, en caso de existir errores, su localización y ajustes fueran fácilmente localizables. Los resultados obtenidos de los entrenamientos **quedaron guardados en la carpeta */results* y los modelos finales almacenados en */models*.**

Las tareas de **preprocesamiento de vídeos, generación de los datasets y las de entrenamiento y evaluación de los modelos fueron automatizadas** también mediante scripts de Python, **reduciendo así la intervención humana y facilitando la reproducibilidad** de los experimentos. Los notebooks, guardados dentro de la carpeta */notebooks*, se utilizaron para **tareas de análisis, visualización e interpretación** de los resultados obtenidos.

El principal **mecanismo** que se utilizó para **garantizar la reproducibilidad** del proyecto **fue el versionado de los conjuntos de datos y las anotaciones**. Cada versión del dataset tiene su correspondiente **fichero *dataset_vx.json*** así como un ***split_vx.json*** donde **se guarda de manera explícita toda la información utilizada en cada versión** en específico. Esto permite **regenerar y reutilizar un set de datos específico que haya sido empleado en experimentos anteriores** aunque se hayan agregado más vídeos en la carpeta de datos.

Adicionalmente **se utilizaron semillas fijas en las diferentes etapas del proyecto**, por ejemplo en la **generación de los splits y los entrenamientos** en TensorFlow y Ultralytics. De esta forma, aunque ciertas operaciones ejecutadas en GPU puedan introducir ligeras modificaciones no reproducibles, **estas medidas permiten reducir de manera considerable la aleatoriedad y mantener la consistencia.**

Todo el proyecto **se desarrolló con Python 3.10 en Linux mediante WSL2** de manera que se pudiera utilizar una **tarjeta gráfica NVIDIA GeForce RTX 3070 Laptop con 8 Gb de VRAM** —dado que en Windows nativo las últimas versiones de TensorFlow no

tiene compatibilidad directa con GPUs—. Además, durante todo el desarrollo **se han utilizado cuatro entornos de ejecución**, cada uno **especializado** en una tarea: **preprocesado de vídeo**, uso de **Label Studio**, entrenamiento de la **CNN con TensorFlow** y el entrenamiento de **YOLO con PyTorch y Ultralytics**. El objetivo de esta separación fue **reducir posibles incompatibilidades entre TensorFlow y PyTorch con la GPU**.

Los requerimientos de **cada entorno** también **quedaron reflejados en la carpeta /environments**, donde se conservó la **lista de librerías con sus respectivas versiones en formato .txt**.

Finalmente, **todo el código junto con las anotaciones, los splits, las configuraciones de los entornos y los modelos finales se encuentran versionados mediante Git y disponibles en el repositorio de GitHub del proyecto, en su respectivo anexo**.

IV. RESULTADOS

IV.I. RESULTADOS DEL DETECTOR YOLO

IV. I. I. COMPARATIVA ENTRE DATASETS

El primer análisis realizado sobre el detector consistió en **comparar** el comportamiento de las arquitecturas **YOLO26n y YOLO26s en función del dataset** utilizado, evaluando ambos modelos en las **cuatro resoluciones** de entrada elegidas: **640, 800, 960 y 1088 píxeles**. Las **Figuras 11 y 12** muestran los resultados de las métricas **Precisión, Recall, mAP@50 y mAP@50:95 sobre el conjunto de test**, tanto en su versión diurna como nocturna, permitiendo analizar en profundidad el **efecto producido al incrementar el número de imágenes** de entrenamiento.

En líneas generales, **los resultados muestran una mejora consistente al utilizar el dataset_v2 en ambas arquitecturas**. Esta mejora resulta **más visible en YOLO26n mientras que en YOLO26s dichas diferencias pasan más desapercibidas**. Esto podría estar relacionado con el **mayor número de parámetros que presenta la versión Small**, que ya tiene unos resultados muy consistentes en la primera versión del dataset, **frente a la Nano**, que posee un mayor rango de mejora inicial. No obstante, **la métrica que presenta una mayor mejoría en ambos escenarios es el mAP@50:95**. Esta es **mucho más exigente que el mAP@50** ya que evalúa la calidad de las detecciones **utilizando diferentes niveles de solapamiento entre la bounding box predicha y la bounding box real**, utilizando diferentes thresholds entre 0.5 y 0.95, y luego **calcula el promedio**.

Si bien es cierto que el **dataset_v2** se mantiene por encima de **dataset_v1** en todas las métricas, la diferencia entre ambas versiones va decreciendo a medida que aumenta la resolución de la imagen. La excepción a esto se encuentra en el mencionado **mAP@50:95**, lo cual indica que la ampliación del dataset sí aportó un beneficio real, no tanto en cuanto a la capacidad de localizar semáforos, sino más bien en la capacidad de delimitar sus **bounding boxes** con mayor precisión.

Esto queda respaldado con la [Figura 13.](#), donde se muestra un mismo semáforo evaluado con ambos modelos y una resolución de 1088 píxeles. Aquí, el modelo YOLO26n entrenado con el **dataset_v1** logra un IoU de 0.604 mientras que con el **dataset_v2** el valor se incrementa hasta 0.813. En el caso de YOLO26s la mejora se observa ya que el IoU aumenta de 0.746 a 0.847. Esto permite corroborar que las **bounding boxes** obtenidas con **dataset_v2** se ciñen más a la anotación real. Aunque se trata de un ejemplo concreto y no nos permite generalizar, sí resulta de interés dado que los recortes elaborados por las **bounding boxes** de YOLO serán las entradas en la CNN.

IV. I. II. COMPARATIVA ENTRE MODELOS

Una vez estudiado el efecto del **dataset** en las distintas configuraciones del detector se debe realizar la **comparativa entre las variantes del modelo**. Los resultados se recogen en las [tablas 5 y 6](#) donde se detallan las cuatro resoluciones para cada modelo en cada versión del dataset, tanto en condiciones diurnas como nocturnas. Sin embargo, teniendo en cuenta los mejores resultados del **dataset_v2**, en este apartado la comparación se centrará únicamente en esta segunda versión de los datos.

En los resultados se observa que la versión **Small** tiene, en general, una mejor **performance** que la versión **Nano**. Estas diferencias existen tanto en el **dataset** diurno como en el **nocturno** y pueden apreciarse con mayor claridad en el **mAP@50:95**. Además de las mencionadas tablas, esto queda recogido en la [Figura 14.](#)

Para el dataset diurno, YOLO26s obtiene valores de 0.7316, 0.7770, 0.8095 y 0.8133 respectivamente para 640, 800, 960 y 1088 píxeles, mientras que YOLOn alcanza 0.6672, 0.7537, 0.7680 y 0.7427. Esta diferencia queda recogida en la [Tabla 7.](#), donde se puede observar que el modelo **Small** es superior al **Nano** con una diferencia promedio de 6.9% en el **mAP@50:95**. Esta ventaja es aún mayor en el **dataset** nocturno. Como muestra la [Tabla 8.](#), YOLO26s obtiene un rendimiento igualmente superior en las cuatro resoluciones, con una mejora promedio del 14.1% con respecto a YOLO26n. Esta diferencia es especialmente notable en resoluciones bajas, con un 22% a 640 píxeles y un 17.8% a 800 píxeles, siendo la menor de tan solo un 3.8% en resoluciones de 1088 píxeles.

Estas diferencias pueden observarse con mayor detalle en la [Figura 15.](#), donde se muestra la evolución de las distintas métricas para ambas arquitecturas —azul para

YOLO26s y rojo para YOLO26n—. Los valores diurnos se mantienen por encima de los valores nocturnos y, en general, **la versión Small queda por encima de la Nano**.

De manera complementaria, en la [Figura 16](#), se recoge cuánto rendimiento se mantiene al pasar del dataset diurno al nocturno y donde valores cercanos a uno significan que no se pierde rendimiento mientras que valores más bajos indican una mayor pérdida de la *performance*. En este caso, el **mAP@50:95** es la métrica que más se ve comprometida durante la noche. Este comportamiento indica que **la falta de luz no afecta tanto a la localización de un semáforo, sino a la precisión con la que se pueden delimitar sus bordes**. Además, se observa que **esta pérdida de rendimiento puede reducirse en ocasiones al aumentar la resolución**, aunque dicha mejora **no es uniforme ni aplica a todas las métricas**.

En resumen, la comparativa realizada muestra **dos tendencias**: por un lado, YOLO26s se presenta **superior de manera global** que YOLO26n, con una **ventaja especialmente relevante en el mAP@50:95 y en condiciones nocturnas**; por otro lado, una **mayor resolución suele ir acompañada de un mayor rendimiento**, aunque **los rendimientos marginales son decrecientes** y no son uniformes en todas las métricas. **Por tanto, el detector final deberá buscar un equilibrio** entre rendimiento, estabilidad y coste computacional.

IV.II. RESULTADOS DEL CLASIFICADOR (CNN)

IV. II. I. RESULTADOS EN VALIDACIÓN

El **primer paso** que se ha realizado para analizar el rendimiento de los modelos clasificadores ha sido **estudiar los resultados sobre los conjuntos de validación**. La **distribución de los distintos datasets** tanto de validación como de test, **quedan registrados en la [Tabla 9](#)**.

La **primera versión consta de 1.022 imágenes**, donde un **55.28%** pertenecen a la clase **Red** y un **43.64%** a la clase **Green**. En la **segunda versión**, se incrementó el número de imágenes hasta **1.905**, lo que supone un **aumento del 86.40%**. Aquí la **distribución de las clases** se mantuvo similar a la primera versión, con un **55.80%** y un **43.15%** respectivamente.

La mencionada **tabla** también **contiene la distribución de los conjuntos de test**, que serán analizados en el siguiente apartado. Para el **set diurno** se registraron **816 imágenes con unos porcentajes de clases Red y Green del 53.55 y 44.85%** respectivamente. Por otra parte, en cuanto al **set nocturno**, se contó con **1119 imágenes con una distribución del 47.90 y 45.04%**.

Una vez realizados los entrenamientos, **se evaluó el rendimiento de cada modelo sobre su respectivo dataset**. Dado que **la salida del clasificador** se corresponde con la

probabilidad de Green, para obtener conclusiones de **métricas como el recall o la especificidad** se ha de tener en cuenta **Green como clase positiva y Red como clase negativa**.

Para este análisis **se ha tomado la decisión metodológica de no utilizar el mismo umbral de decisión en ambos modelos**. Esto se debe a que **cada CNN puede estar generando diferentes distribuciones de probabilidad para cada clase** y evaluar ambos clasificadores con el mismo threshold **generaría el riesgo** de que algún modelo —o ambos— pudiera **no encontrarse en las condiciones óptimas de operatividad**. Por tanto, **cada CNN cuenta con un threshold específico** que se corresponde a su **punto de Youden**. Dichos valores se pueden observar registrados en la [Figura 17](#), siendo **0.0465 para el modelo entrenado con dataset_v1 y 0.089 para dataset_v2**.

Los resultados obtenidos al aplicar dichos thresholds quedan recogidos en la [Tabla 10](#), y más gráficamente en la [Figura 18](#), mostrando una **mejoría generalizada en la segunda CNN**. No obstante, **analizar un solo punto de corte limita las posibles conclusiones**. Por esta razón **se estudió también la evolución que presentan dichas métricas para diferentes valores del umbral de decisión**. Tal y como se observa en la [Figura 19](#), el modelo entrenado con dataset_v2 presenta valores máximos de Accuracy, Precision, Recall, F1 y Specificity durante prácticamente todo el intervalo de thresholds utilizado. Sin embargo, el modelo entrenado con dataset_v1 muestra cierta **degradación**, más concretamente en métricas como el Accuracy, Recall y F1.

Estas **diferencias** se aprecian mejor **al observar los falsos negativos**, recogidos en la [Figura 20](#), donde **incrementar el threshold genera que, para el primer modelo, estos aumenten rápidamente**. En cambio, **la segunda versión del modelo los mantiene constantes**, respaldando el argumento de que **el segundo modelo permite ser más conservador sin comprometer su capacidad de detectar semáforos en estado Green**. Esto también **puede entenderse mejor si se observa la Figura 21**, donde se pueden ver las **distribuciones de probabilidad por clase** para ambas versiones del modelo. En dataset_v1 —izquierda— existen detecciones del estado Green con probabilidades mucho más bajas que con dataset_v2 —derecha—, donde la **separación de clases es mucho más clara**.

No obstante, **para la versión de test solo se utilizó el punto de Youden de dataset_v1**. Esto se debe a que, para este modelo, **dicho umbral se debe a la existencia de semáforos con estado Green con probabilidades bajas** y aumentarlo supondría **sacrificar falsos negativos**. Sin embargo, **para la segunda versión**, y aunque el punto de Youden se sitúe en 0.089, **existe un amplio margen de maniobra** donde podría situarse el threshold **sin que estos falsos negativos se vean comprometidos**.

Por dicha razón, y dado que **el coste de los errores no es simétrico** en este proyecto —perjudica más un falso positivo que un falso negativo—, **se establece un threshold de 0.80**. Los resultados quedan recogidos en la [Tabla 11](#), donde se observa un

ligero empeoramiento de ciertas métricas aunque las **diferencias son prácticamente nulas**.

IV. II. II. RESULTADOS EN TEST

Una vez analizados los modelos para el conjunto de validación y definidos los thresholds a utilizar, **el siguiente paso será evaluar dichos clasificadores con los conjuntos de test**, siendo estos completamente **independientes y grabados única y exclusivamente para esta labor**.

Los **resultados** obtenidos están recogidos en la [Tabla 12](#) y en la [Figura 22](#). En el conjunto de **datos diurnos** el clasificador que fue entrenado con **dataset_v2** **presenta un rendimiento superior** en todas las métricas, **mejorando el accuracy en un 2.45%, la precisión en 3.24%, el recall en 2.18% y el F1-Score y la especificidad en un 2.71 y 2.66% respectivamente**. Por su parte, los **falsos positivos** se ven reducidos **de 13 a 1** y los **falsos negativos de 9 a 1**. En el conjunto nocturno, ambos clasificadores presentan unos resultados perfectos, sin registrar falsos positivos ni falsos negativos.

No obstante, de la misma manera que se hizo en con el conjunto de validación, los **modelos deben analizarse utilizando** un abanico de **umbrales de decisión** pues, de limitar el análisis a uno, **las conclusiones estarían muy limitadas**. Así pues, la **evolución** de las mencionadas **métricas quedan recogidas en las figuras 23 y 24**.

En cuanto al **dataset nocturno**, **ambos modelos tienen un rendimiento parejo** con **diferencias menores** en thresholds bajos aunque **dichas diferencias convergen** a medida que este se incrementa. De hecho, los **falsos positivos son nulos en el modelo entrenado con el primer dataset** mientras que **desaparecen rápidamente para el modelo que utiliza dataset_v2**.

En cambio, al analizar los **resultados diurnos** **sí se observan diferencias mayores** entre ambos clasificadores. **A medida que aumenta el threshold, métricas como la precisión y la especificidad mejoran y convergen**. Sin embargo, **otras como el accuracy, el recall y el F1 muestran una clara y progresiva degradación del rendimiento del modelo basado en dataset_v1** mientras que la segunda versión de este mantiene resultados extraordinarios con thresholds superiores a 0.5.

Ahondando en los **falsos negativos y falsos positivos**, **el primer modelo parece tener inicialmente valores más bajos de falsos positivos**, aunque la diferencia se reduce progresivamente a medida que se incrementa el threshold. Esto no ocurre con los **falsos negativos**, donde **el segundo modelo muestra un comportamiento superior a lo largo de todo el intervalo de thresholds**. Esto es un indicador de que **la segunda versión del clasificador es más estable** ante cambios en el umbral de decisión.

Los **errores en la clasificación y su distribución según P(Green)** pueden observarse en la [Figura 25](#). En la **primera versión** del clasificador **aparecen tanto falsos negativos como falsos positivos** con **probabilidades relativamente bajas**. En cambio, la

segunda CNN tan solo presenta **dos errores**, uno de cada tipo: un falso negativo con probabilidad 0.658 y un falso positivo con una probabilidad de 0.884, **presentando una separación de clases más limpia**.

Finalmente, en la [Figura 26](#) se muestran las **distribuciones de P(Green)** según la clase en ambas CNN. En la primera, más concretamente en el dataset diurno, se observa una mayor dispersión de las predicciones con muestras de la clase Green cuyas probabilidades se solapan con las de la clase Red. Por esta razón, la capacidad de este modelo de funcionar en umbrales más altos se ve limitada. Sin embargo, en la segunda CNN se observan **distribuciones más separadas** donde valores altos de P(Green) son, efectivamente, muestras de clase Green y valores bajos se corresponden con semáforos cuya clase es Red. Esto es un signo de una mayor estabilidad y permite utilizar un criterio más conservador en cuanto al umbral de decisión, que para este experimento se ha situado por debajo del punto de Youden teórico de test.

No obstante, **ambos modelos presentan unos resultados excelentes en la clasificación nocturna**, siendo incluso superior el modelo entrenado sobre dataset_v1.

Los dos errores cometidos por la CNN entrenada con dataset_v2 están registradas en la [Figura 27](#), y son un falso negativo y un falso positivo. Además, en la [Figura 28](#) se muestran los *feature maps* obtenidos mediante una metodología Grad-CAM++. Para el primer caso —izquierda—, la CNN parece recoger información ajena al semáforo; en el segundo caso —derecha—, no parece ser capaz de captar información relevante para la predicción. No obstante, son casos aislados y suponen un porcentaje ínfimo de entre el conjunto completo donde la CNN sí es capaz de recoger información relevante de la imagen para realizar las predicciones, tal y como se recoge en la [Figura 29](#), donde se muestran ejemplos de semáforos correctamente clasificados junto con sus Grad-CAM++.

IV.III. INTEGRACIÓN DE LOS MODELOS

Tras realizar la evaluación individual de los modelos, estos fueron integrados dentro de un mismo flujo mediante un prototipo desarrollado en Streamlit con asistencia de Codex. De esta forma se puede cargar un vídeo y ver cómo funcionan en conjunto ambos modelos. Primero, YOLO detectará el semáforo cuya área sea la mayor, pues se entiende que este será el más cercano y por tanto el que regule el cruce más próximo. A través de las bounding boxes, genera el recorte y se lo envía a la CNN, la cual se encarga de realizar la clasificación del estado mediante P(Green) y catalogando los semáforos apagados como Red.

La interfaz está recogida en la [Figura 30](#), y muestra funciones como: la selección del modelo y resolución, selección de confianza YOLO y el *threshold* de la CNN, la posibilidad de cambiar el brillo, contraste o saturación del vídeo, la capacidad de

mostrar la **previsualización** durante el procesado, **mantener o no el audio** del vídeo y seleccionar la **resolución final del vídeo**.

Para este ejemplo **se utilizó YOLO26s a 1088 píxeles de resolución y la CNN en formato ONNX**. Esto es porque, durante el desarrollo, **cada fase fue realizada en un entorno diferente**: el **clasificador** se desarrolló y entrenó **utilizando TensorFlow** mientras que para el **detector** se **utilizó** un entorno basado en **Ultralytics y PyTorch**. Y **para que ambos pudiesen ejecutarse simultáneamente de manera sencilla**, se mantuvo el entorno utilizado para YOLO y por tanto se convirtió el modelo clasificador a ONNX.

En el vídeo elegido **se tomaron 796 fotogramas, de los cuales 370 contenían una detección válida enviada al clasificador, siendo los resultados 265 para la clase Red y 105 para la clase Green**. El procesado **se pudo completar en 17.5 segundos**, procesando un total de **45.56 fotogramas por segundo**. Además, en la figura se añaden las latencias por modelo y el tiempo total de ejecución, con un total de **12.07 segundos para el detector y 0.44 segundos para el clasificador** —aproximadamente un **96%** del tiempo consumido pertenece a YOLO y un **4%** para la CNN—.

Se adjuntan **ejemplos del funcionamiento** en las **Figuras 31 —semáforo nocturno apagado—, 32 —semáforo nocturno correctamente clasificado— y 33 —cruce múltiple con tan solo el semáforo más cercano detectado—**.

En resumen, **este prototipo permite comprobar el rendimiento conjunto y encadenado de ambos modelos y demuestra tanto su utilidad como su eficacia**. Aun así, **su finalidad es únicamente de carácter demostrativo y no debe entenderse como una aplicación funcional ni una solución fiable para su uso como herramienta de asistencia visual**.

V. CONCLUSIONES Y TRABAJO FUTURO

El objetivo principal del trabajo de fin de máster ha sido, una vez más, **crear un sistema de computer vision capaz de detectar y clasificar semáforos peatonales** y estudiar sus posibilidades **como herramienta de asistencia para personas con discapacidad visual** sobre **un dataset propio compuesto por 182 vídeos, todos ellos grabados desde el punto de vista de un peatón**, entrenando para tal causa tanto **modelos de la familia YOLO26 —encargados de la detección— como una red convolucional** —para la clasificación del estado de los semáforos—.

Tomando en consideración el tamaño del dataset, **los modelos entrenados sobre dataset_v2 han presentado resultados más robustos con mejoras generalizadas** sobre la **mayoría de métricas**. Además, se observó que **una mayor resolución, en general,**

viene acompañada de unos mejores resultados y facilita la **detección de objetos de menor tamaño**.

Por ejemplo, **para el detector, ampliar el dataset** supuso **mejorar la capacidad de delimitar las bounding boxes, con un impacto** significativo en el **mAP@50:95**. YOLO26s fue superior a YOLO26n en la mayoría de los escenarios, más concretamente en entornos nocturnos, aunque con el correspondiente incremento en el coste computacional.

Con respecto a la CNN, se observó una tendencia similar, con **métricas más robustas y una clasificación más limpia en la segunda versión**. También se observó que, contrariamente a lo que ocurre con el detector, **la capacidad predictiva es superior de noche**.

Por último, **el prototipo creado con Streamlit demuestra que ambos modelos pueden trabajar juntos**, a pesar de que **la mayoría de la capacidad computacional** va dirigida al **detector, quien podría suponer el cuello de botella principal**.

En cuanto a las líneas futuras de investigación se plantea lo siguiente:

1. Desarrollar una **aplicación móvil** para **testear el rendimiento** de los modelos **en un hardware más reducido**.
2. **Ampliar el dataset** a nuevas **localizaciones** tanto **nacionales como internacionales** para mejorar la capacidad de generalización de los modelos.
3. Estudiar **técnicas de optimización**, como por ejemplo la **cuantización de los modelos YOLO o las Depthwise Separable Convolutions**.
4. Implementar **tracking de los semáforos** así como **redes recurrentes** para analizar el estado de estos durante varios frames consecutivos.
5. **Estudiar otros elementos** presentes a la hora de cruzar una calle, como los **pasos de peatones o coches en movimiento**.
6. Realizar pruebas en las CNN utilizando diferentes resoluciones de imagen.

VI. REFERENCIAS

1. Armengol, J. [@armengoljonatan]. (2026, 20 de agosto). *En este vídeo os muestro un semáforo en pleno funcionamiento que, perfectamente, podría convertirse en una trampa mortal para las personas ciegas* [Video]. Instagram. [Link](#)
2. Behrendt, K., Novak, L., & Botros, R. (2017). A deep learning approach to traffic lights: Detection, tracking, and classification. In 2017 IEEE International Conference on Robotics and Automation (ICRA) (pp. 1370–1377). IEEE. <https://doi.org/10.1109/ICRA.2017.7989163>
3. Cheng, R., Wang, K., Yang, K., Long, N., Bai, J., & Liu, D. (2018). Real-time pedestrian crossing lights detection algorithm for the visually impaired. *Multimedia Tools and Applications*, 77(16), 20651–20671. <https://doi.org/10.1007/s11042-017-5472-5>
4. El-Taher, F. E.-Z., Taha, A., Courtney, J., & Mckeever, S. (2021). A systematic review of urban navigation systems for visually impaired people. *Sensors*, 21(9), 3103. <https://doi.org/10.3390/s21093103>
5. Ismail, M. I. M., & Mousa, M. A. A. (2025). Employing computer vision on a smartphone to help the visually impaired cross the road. *Proceedings of the AAAI Symposium Series*, 6(1), 227–234. <https://doi.org/10.1609/aaaiss.v6i1.36057>
6. Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F. C., & Kalfaoglu, M. E. (2026). Ultralytics YOLO26: Unified real-time end-to-end vision models [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2606.03748>
7. ONCE. (s. f.-a.). *Datos de personas afiliadas a la ONCE*. Recuperado el 23 de agosto de 2026. [Conoce los datos anuales de nuestros afiliados - Web ONCE](#)
8. ONCE. (s. f.-b.). *Requisitos de afiliación a la ONCE*. Recuperado el 23 de agosto de 2026. [Requisitos de afiliación](#)
9. ONCE. (s. f.-c.). *Desplazamiento autónomo*. Recuperado el 23 de agosto de 2026. [Desplazamiento autónomo](#)

10. International Agency for the Prevention of Blindness. (s. f.). Spain. IAPB Vision Atlas. Recuperado el 23 de agosto de 2026, [The Value of Vision: The case for investing in eye health.](#)
11. Bourne R, Steinmetz J, Flaxman S et al. Trends in prevalence of blindness and distance and near vision impairment over 30 years: an analysis for the Global Burden of Disease Study. The Lancet Global Health, 2020; 9, e130-e143. [https://doi.org/10.1016/S2214-109X\(20\)30425-3](https://doi.org/10.1016/S2214-109X(20)30425-3)
12. Prieto González, D. (2025, diciembre). Discapacidad visual en cifras: El mapa global 2026. NaviLens. <https://www.navilens.com/blog/discapacidad-visual-cifras-globales>
13. Barlow, J. M., Bentzen, B. L., & Bond, T. (2005). Blind pedestrians and the changing technology and geometry of signalized intersections: Safety, orientation, and independence. *Journal of Visual Impairment & Blindness*, 99(10), 587-598. <https://doi.org/10.1177/0145482X0509901003>
14. Organización Mundial de la Salud. (2026, 10 de febrero). Discapacidad visual y ceguera. [Discapacidad visual y ceguera](#)
15. Ultralytics. (2026a). Ultralytics YOLO26. Recuperado el 29 de agosto de 2026, de <https://docs.ultralytics.com/models/yolo26/>
16. Ultralytics. (2026b). YOLO hyperparameter tuning. Recuperado el 29 de agosto de 2026, de <https://docs.ultralytics.com/guides/hyperparameter-tuning/>
17. Ultralytics. (2026c). YOLO26 training recipe. Recuperado el 29 de agosto de 2026, de <https://docs.ultralytics.com/guides/yolo26-training-recipe>
18. Ultralytics. (2026d). Customizing trainer. Recuperado el 30 de agosto de 2026, de <https://docs.ultralytics.com/guides/custom-trainer/>
19. Yu, S., Lee, H., & Kim, J. (2019). LYTNet: A convolutional neural network for real-time pedestrian traffic lights and zebra crossing recognition for the visually impaired. In *Computer Analysis of Images and Patterns* (pp. 259-270). Springer. https://doi.org/10.1007/978-3-030-29888-3_21

VII. ANEXOS

VIII.I. FIGURAS

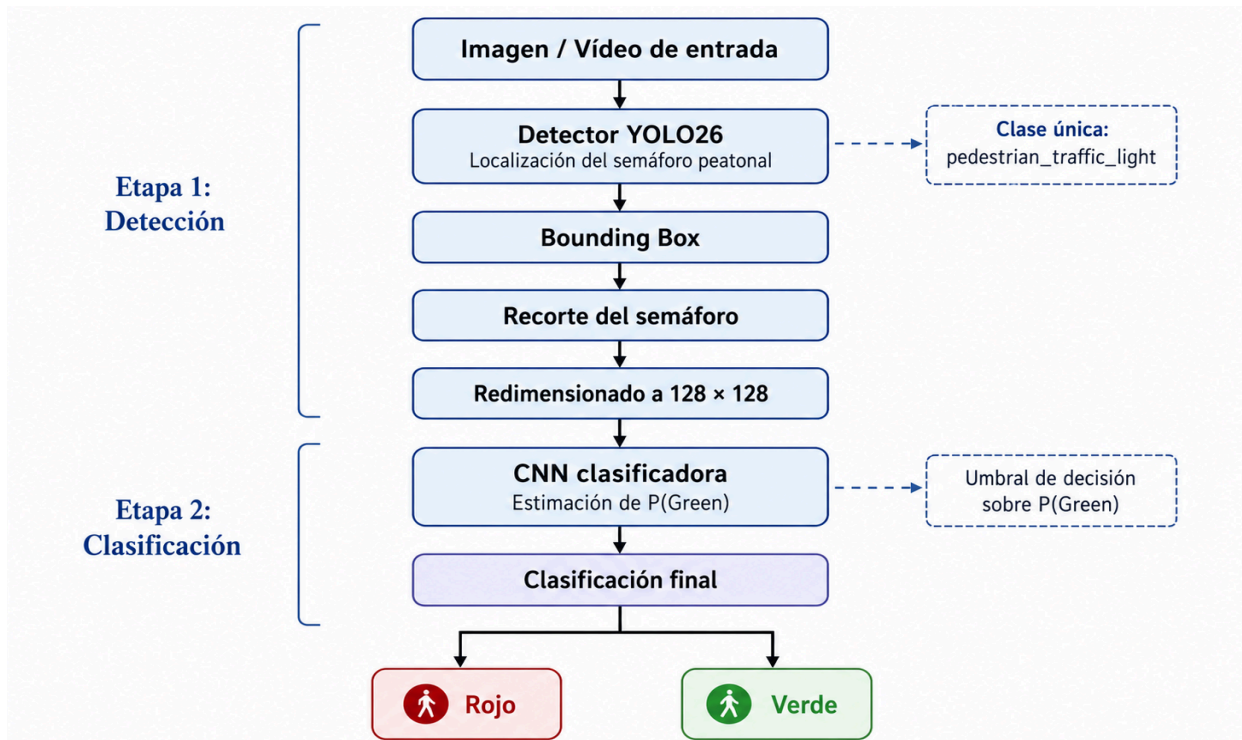


Figura 1. Arquitectura general del sistema.

Fuente: elaboración propia con asistencia de inteligencia artificial generativa. [\(Volver\)](#)



Figura 2. Comparación entre la imagen original y la detección realizada por YOLO26.

Fuente: elaboración propia. [\(Volver\)](#)

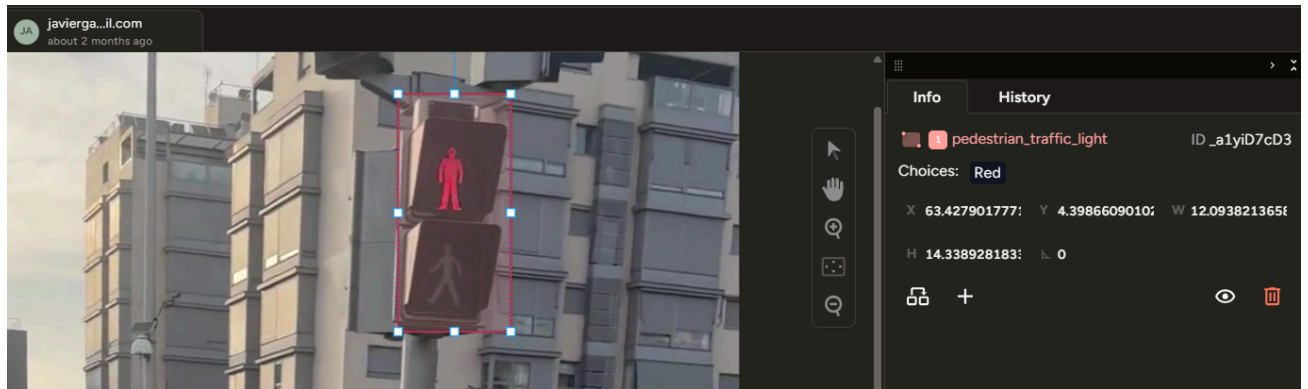


Figura 3. Etiquetado de semáforo peatonal utilizando Label Studio.
Fuente: elaboración propia. ([Volver](#))



Figura 4. Etiquetado de semáforo peatonal nocturno utilizando Label Studio.
Fuente: elaboración propia. ([Volver](#))

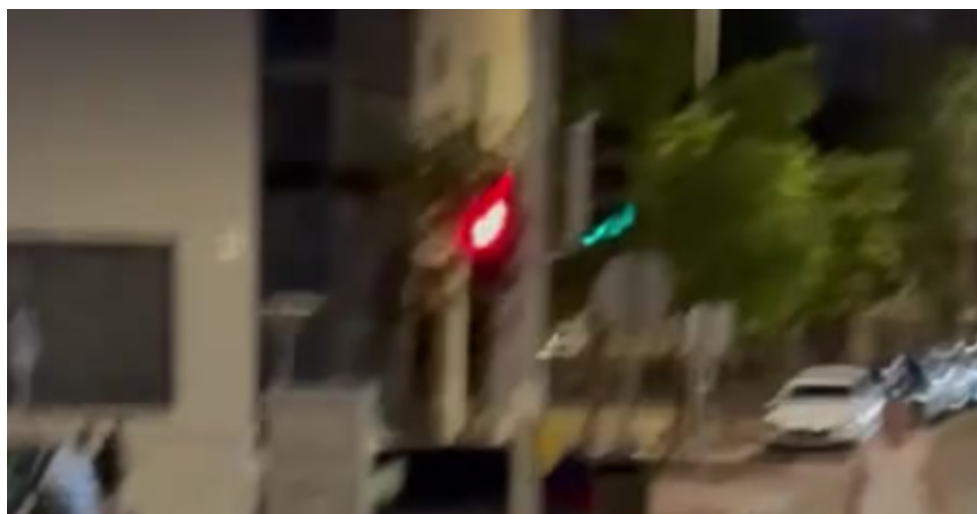


Figura 5. Semáforo nocturno no etiquetable.
Fuente: elaboración propia. ([Volver](#))

```

{
  "train": [
    "000001",
    "000004",
    "000005",
    "000006",
    "000008",
    "000009",
    "000010",
    "000013",
    "000014",
    "000015",
    "000016",
    "000018",
    "000019",
    "000020",
    "000021",
    "000022",
    "000024",
    "000025",
    "000026",
    "000029",
    "000030",
    "000032"
  ],
  "val": [
    "000002",
    "000003",
    "000007",
    "000011",
    "000012",
    "000017",
    "000023",
    "000027",
    "000028",
    "000031"
  ]
}

```

Figura 6. Contenido del fichero split_v1.json.

Fuente: elaboración propia. [\(Volver\)](#)

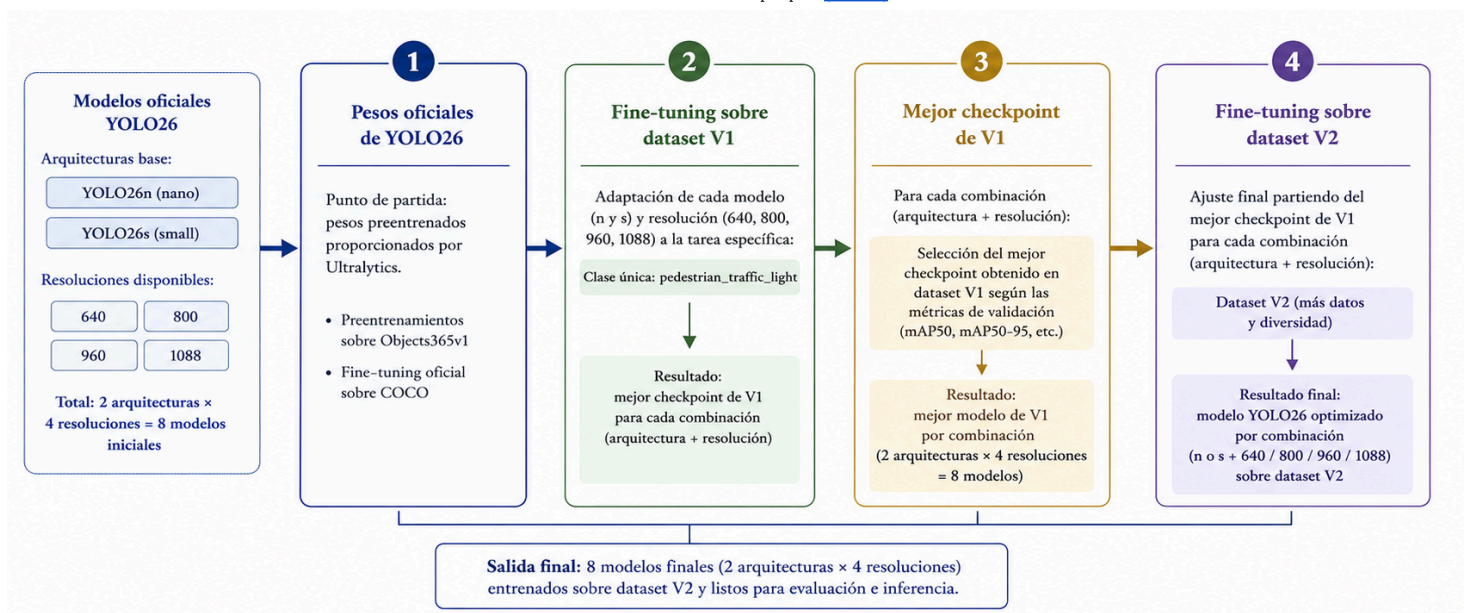


Figura 7. Proceso de fine tuning de los modelos YOLO.

Fuente: elaboración propia con asistencia de inteligencia artificial generativa. [\(Volver\)](#)

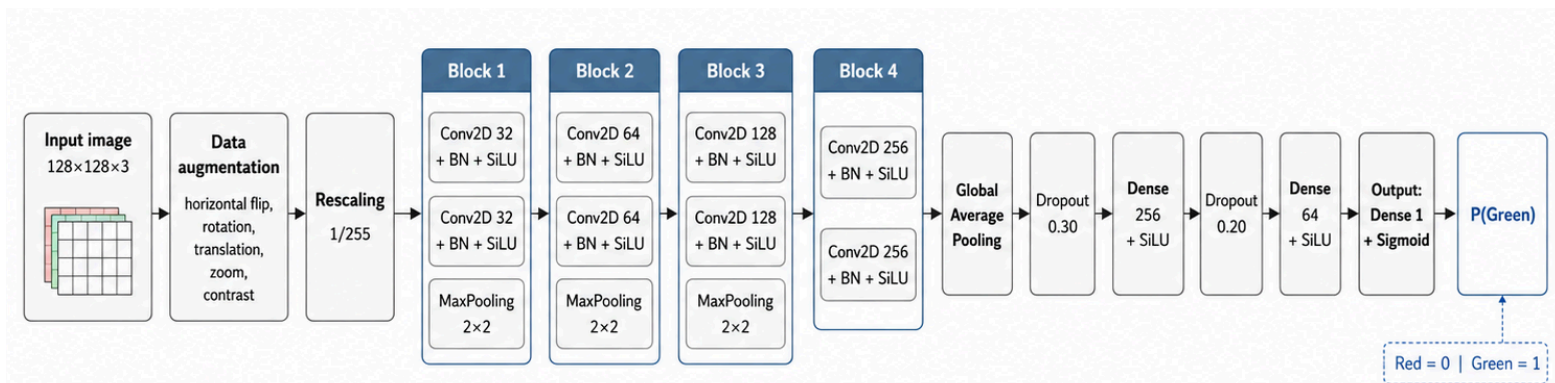


Figura 8. Arquitectura de la CNN para la clasificación del estado de los semáforos.

Fuente: elaboración propia con asistencia de inteligencia artificial generativa. [\(Volver\)](#)

```
data_augmentation = tf.keras.Sequential([

    tf.keras.layers.RandomFlip(
        mode="horizontal",
    ),

    tf.keras.layers.RandomRotation(
        factor=0.03,
    ),

    tf.keras.layers.RandomTranslation(
        height_factor=0.05,
        width_factor=0.05,
    ),

    tf.keras.layers.RandomZoom(
        height_factor=(-0.15, 0.10),
        width_factor=(-0.15, 0.10),
    ),

    tf.keras.layers.RandomContrast(
        factor=0.20,
    ),

])
```

Figura 9. Data Augmentation en el entrenamiento de la CNN.

Fuente: elaboración propia. [\(Volver\)](#)

```

lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=3,
    min_lr=1e-6,
    verbose=1,
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True,
)

checkpoint = tf.keras.callbacks.ModelCheckpoint(
    filepath=RESULTS_DIR / "best.keras",
    monitor="val_loss",
    mode="min",
    save_best_only=True,
    verbose=1,
)

```

Figura 10. Callbacks de la CNN.

Fuente: elaboración propia. [\(Volver\)](#)

YOLO26n — Performance according to image size

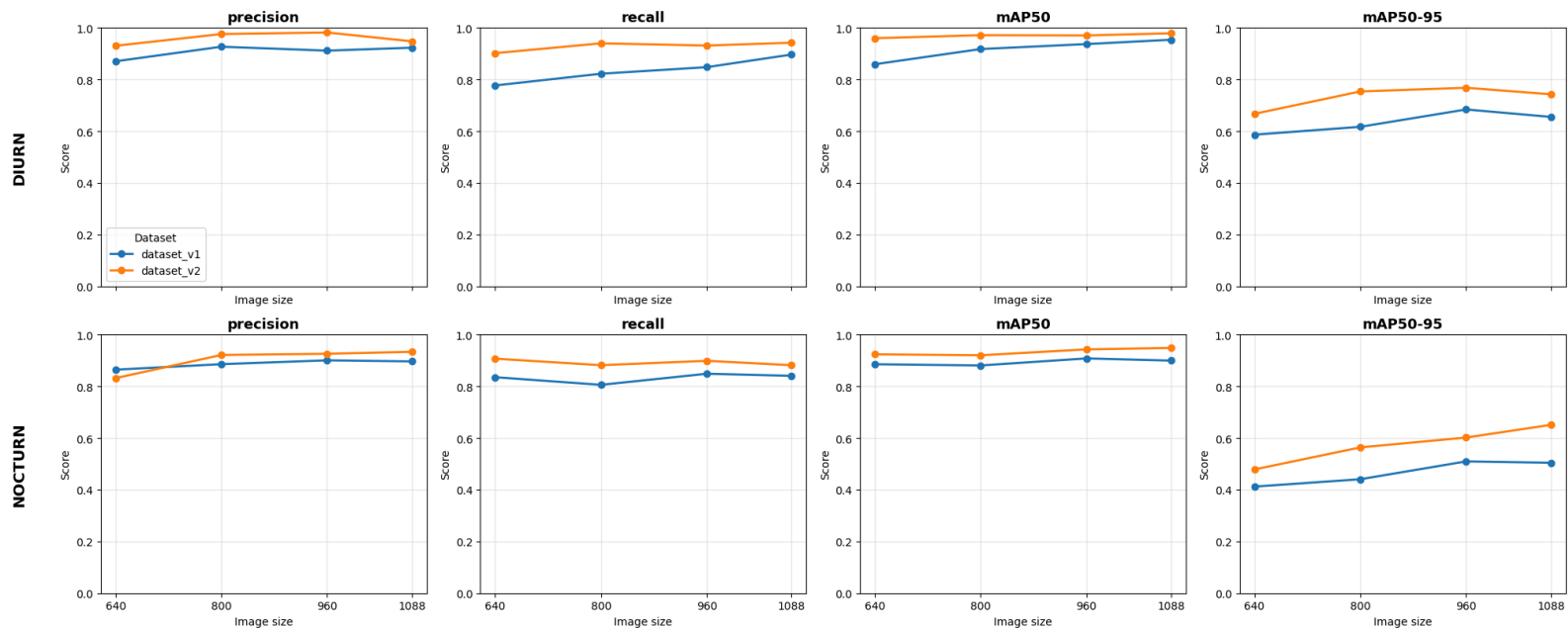


Figura 11. Resultados sobre test de YOLO26n según dataset utilizado, resolución y condiciones de luz.

Fuente: elaboración propia. [\(Volver\)](#)

YOLO26s — Performance according to image size

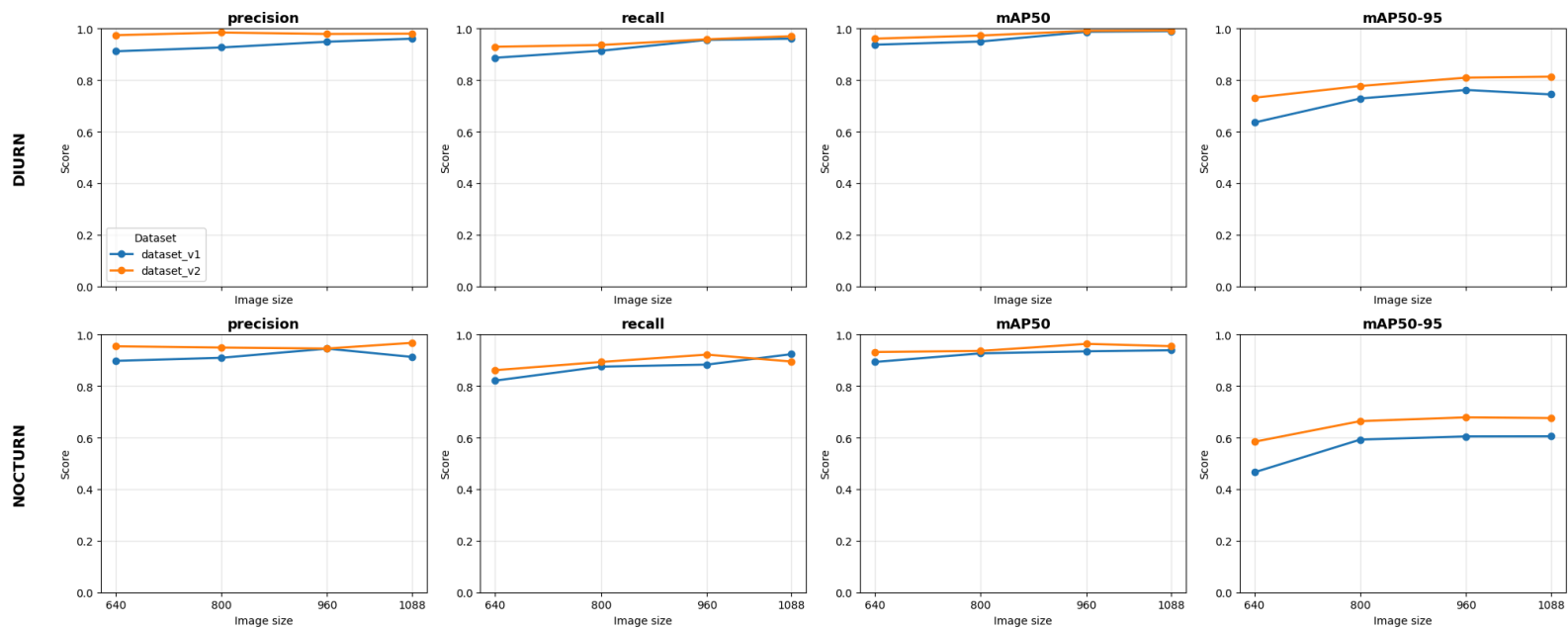


Figura 12. Resultados sobre test de YOLO26s según dataset utilizado, resolución y condiciones de luz.

Fuente: elaboración propia. [\(Volver\)](#)



Ground truth Predicción

Figura 13. Bounding boxes e IoU según dataset para YOLOn y YOLOs 1088 píxeles y threshold 0.1.

Fuente: elaboración propia. [\(Volver\)](#)

YOLO26 — Dataset v2: performance according to image size

DIURN

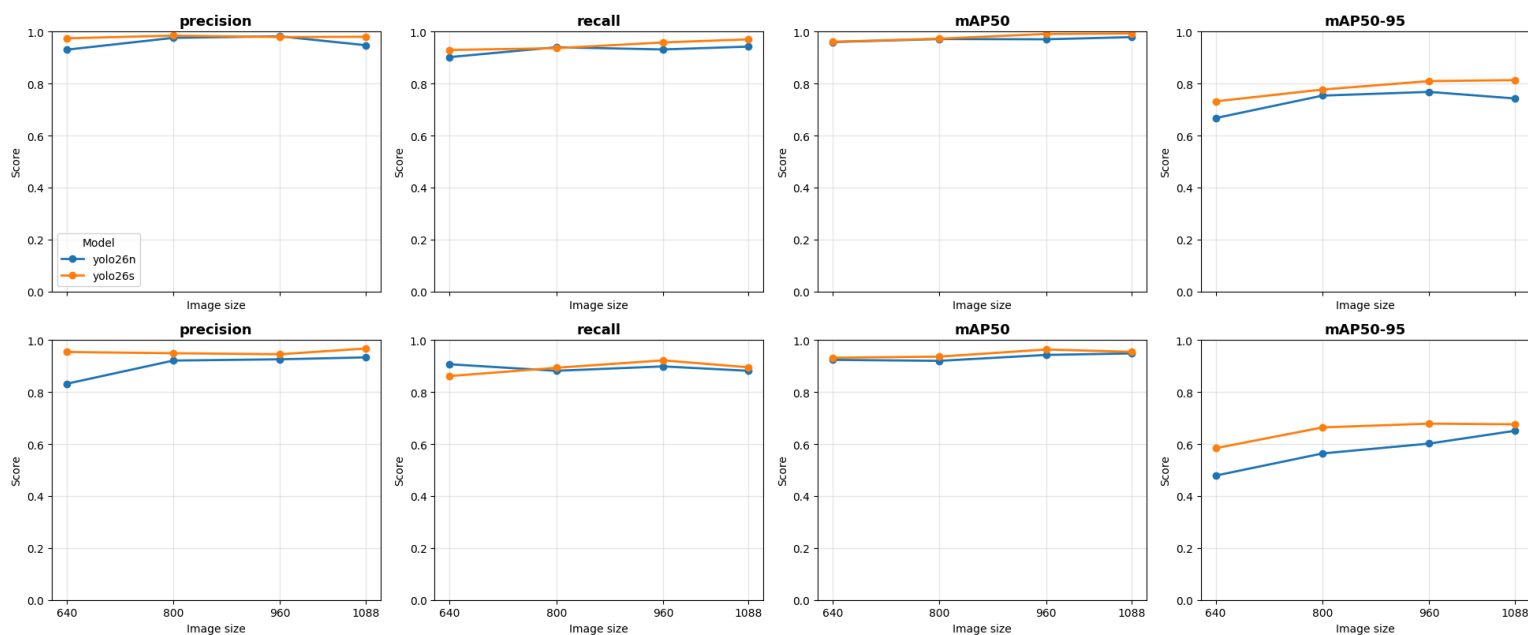


Figura 14. Resultados sobre test según modelo YOLO26 utilizado, resolución y condiciones de luz en dataset_v2.

Fuente: elaboración propia. [\(Volver\)](#)

YOLO26 — Dataset v2: performance by image size and test condition

—●— yolo26n - diurn —●— yolo26n - nocturn —●— yolo26s - diurn —●— yolo26s - nocturn

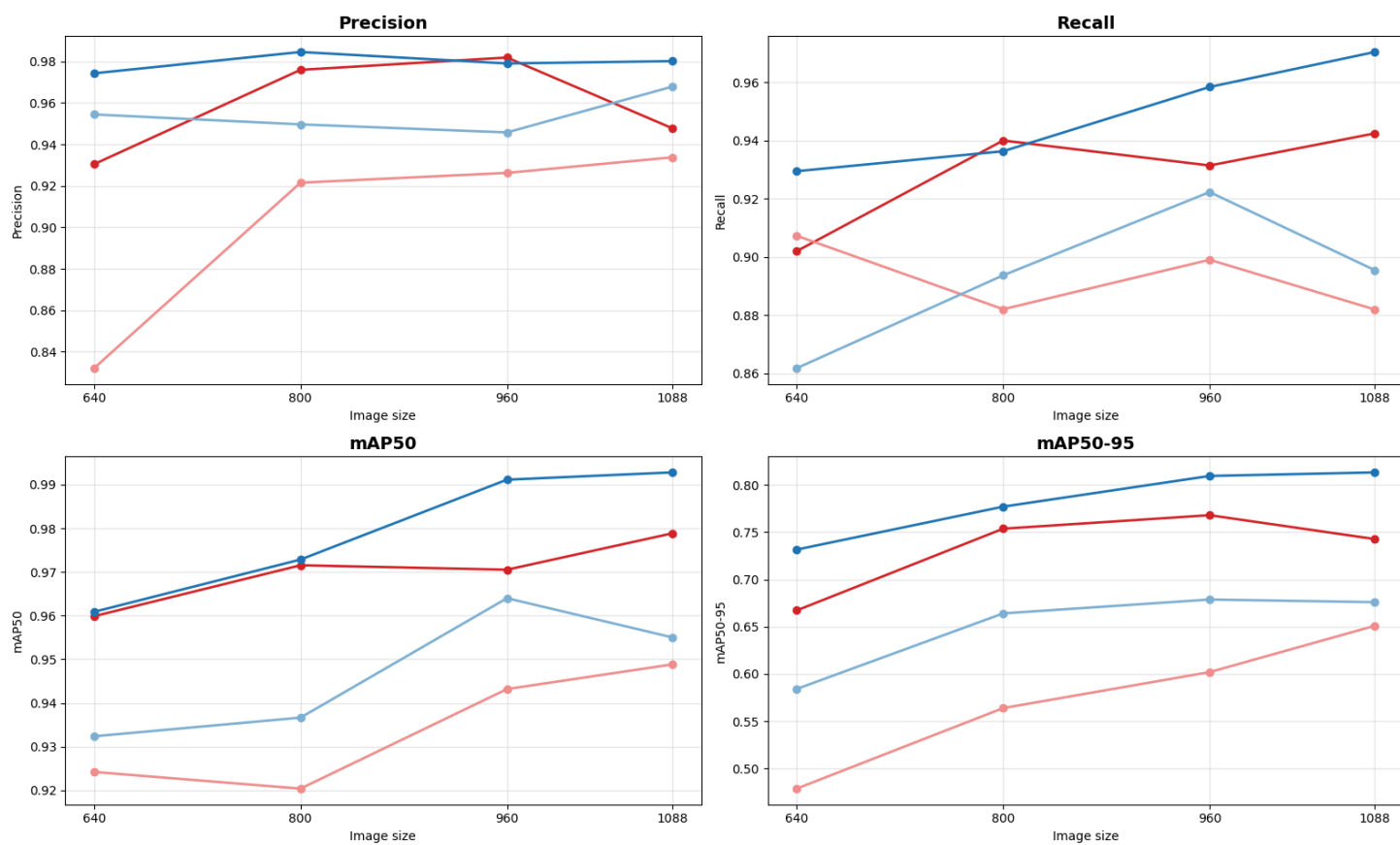


Figura 15. Evolución de métricas según resolución y modelo para dataset_v2.

Fuente: elaboración propia. [\(Volver\)](#)

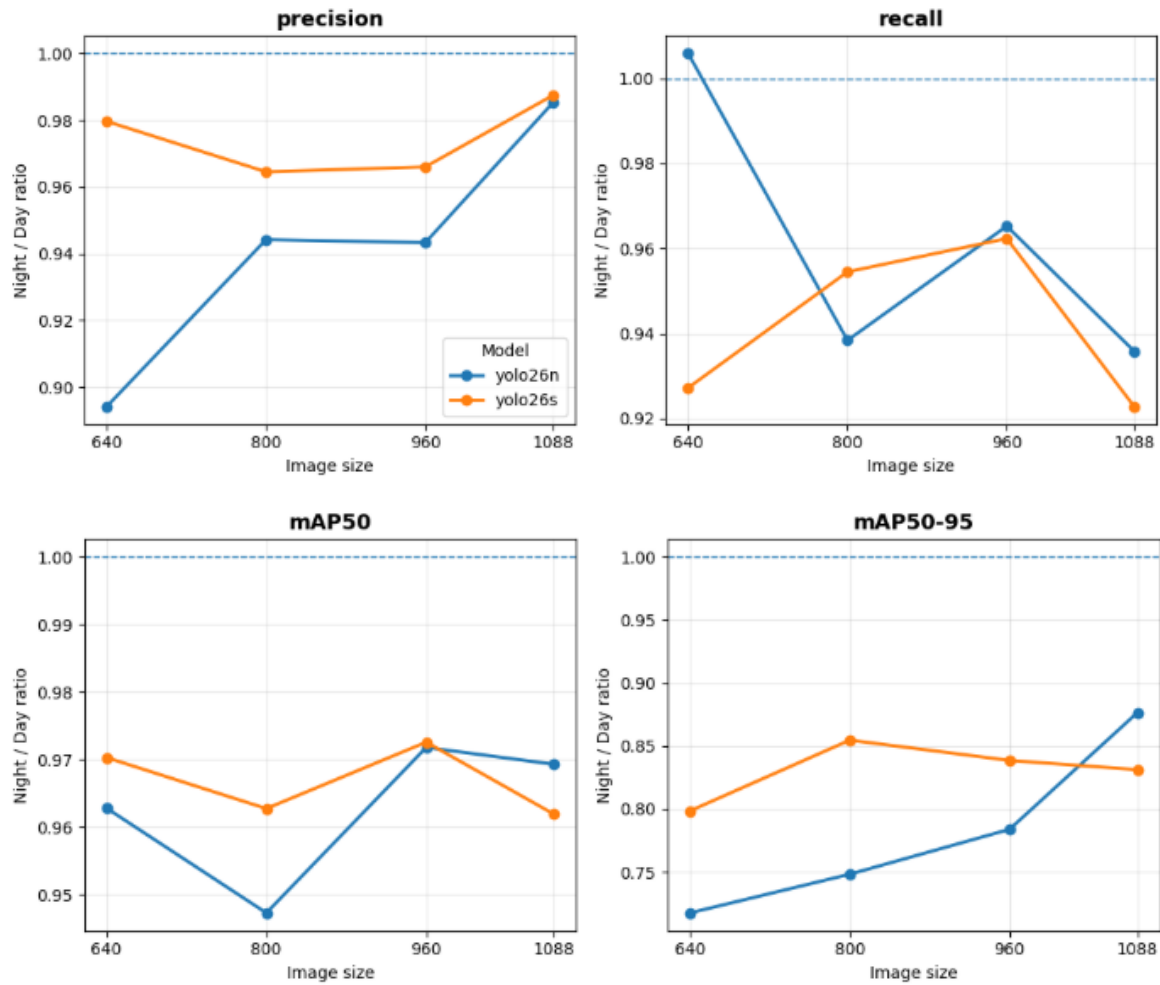


Figura 16. Relación entre el rendimiento diurno y nocturno de YOLO26 Nano y Small según resolución.

Fuente: elaboración propia. [\(Volver\)](#)

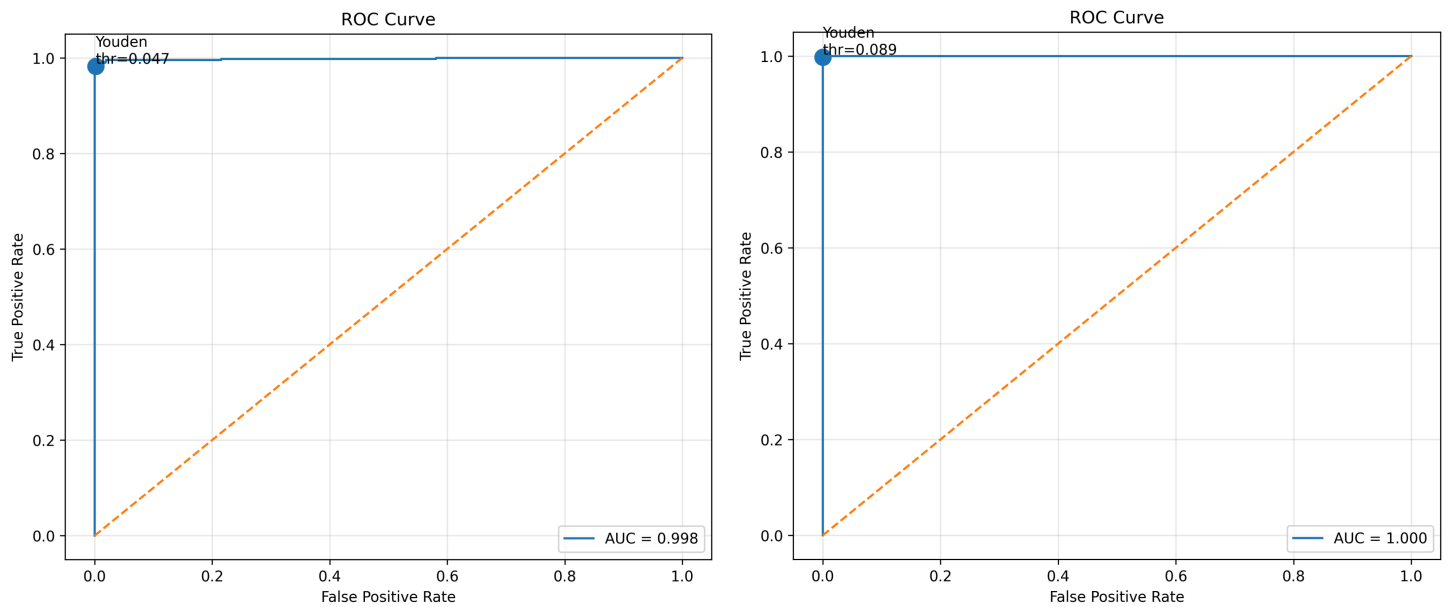


Figura 17. ROC AUC de las CNN. Izquierda: dataset_v1; derecha: dataset_v2.

Fuente: elaboración propia. [\(Volver\)](#)

CNN Classifier — Validation Performance

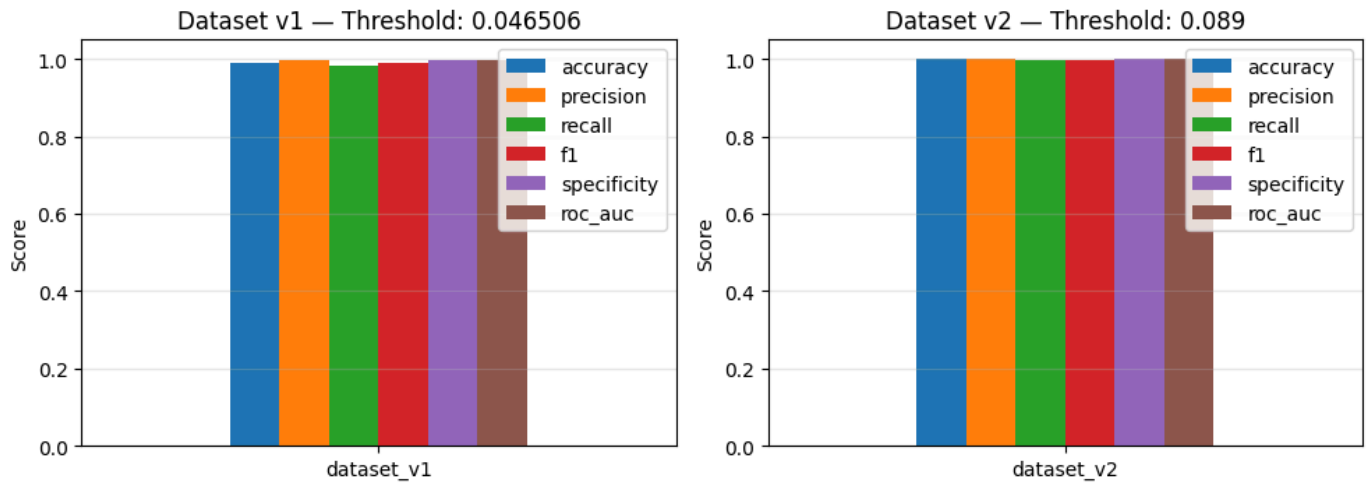


Figura 18. Resultados CNN según versión del dataset utilizando Youden Threshold.

Fuente: elaboración propia. [\(Volver\)](#)

CNN Classifier Performance by Decision Threshold

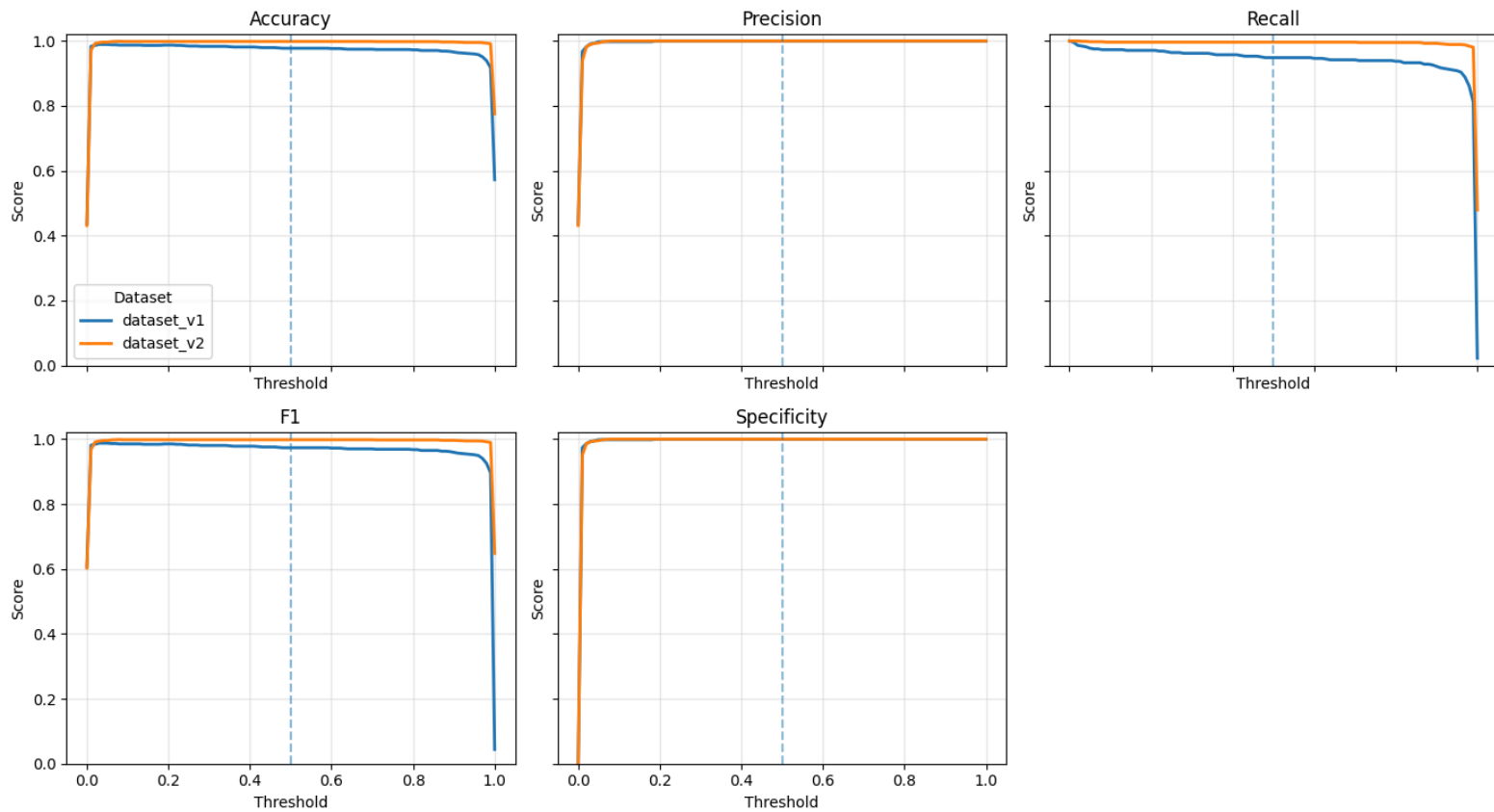


Figura 19. Evolución de métricas en CNN según versión del dataset y threshold.

Fuente: elaboración propia. [\(Volver\)](#)

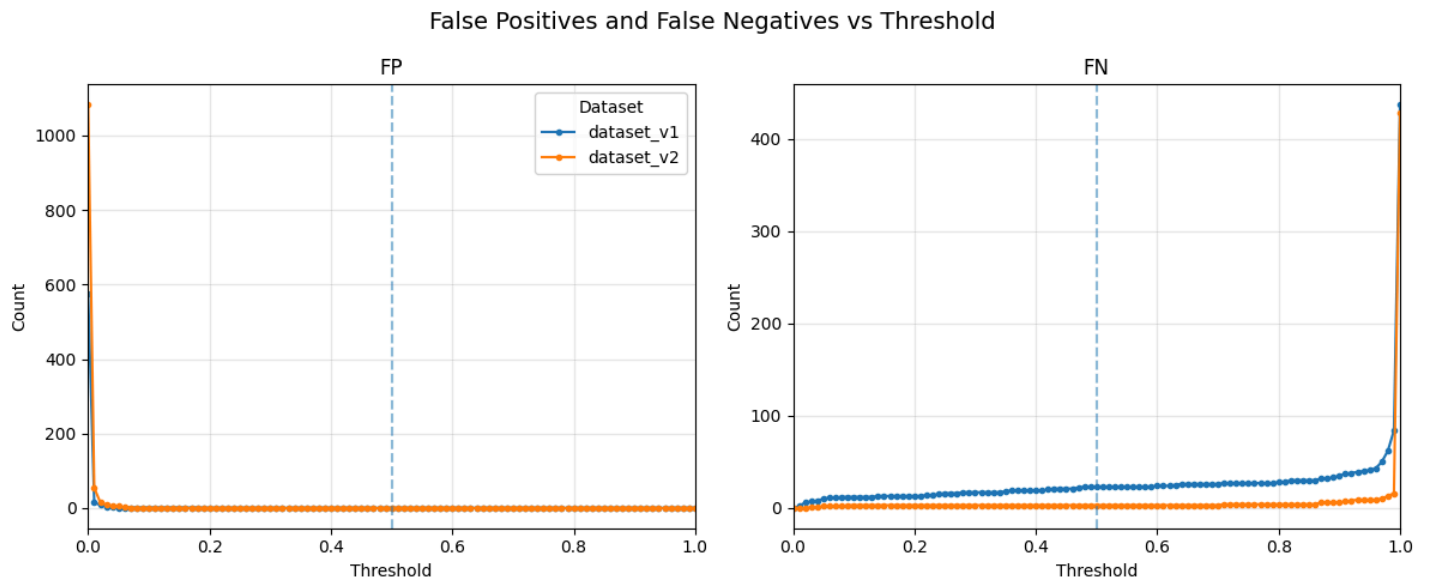


Figura 20. False Positives and False Negatives en CNN según versión del dataset y threshold.

Fuente: elaboración propia. [\(Volver\)](#)

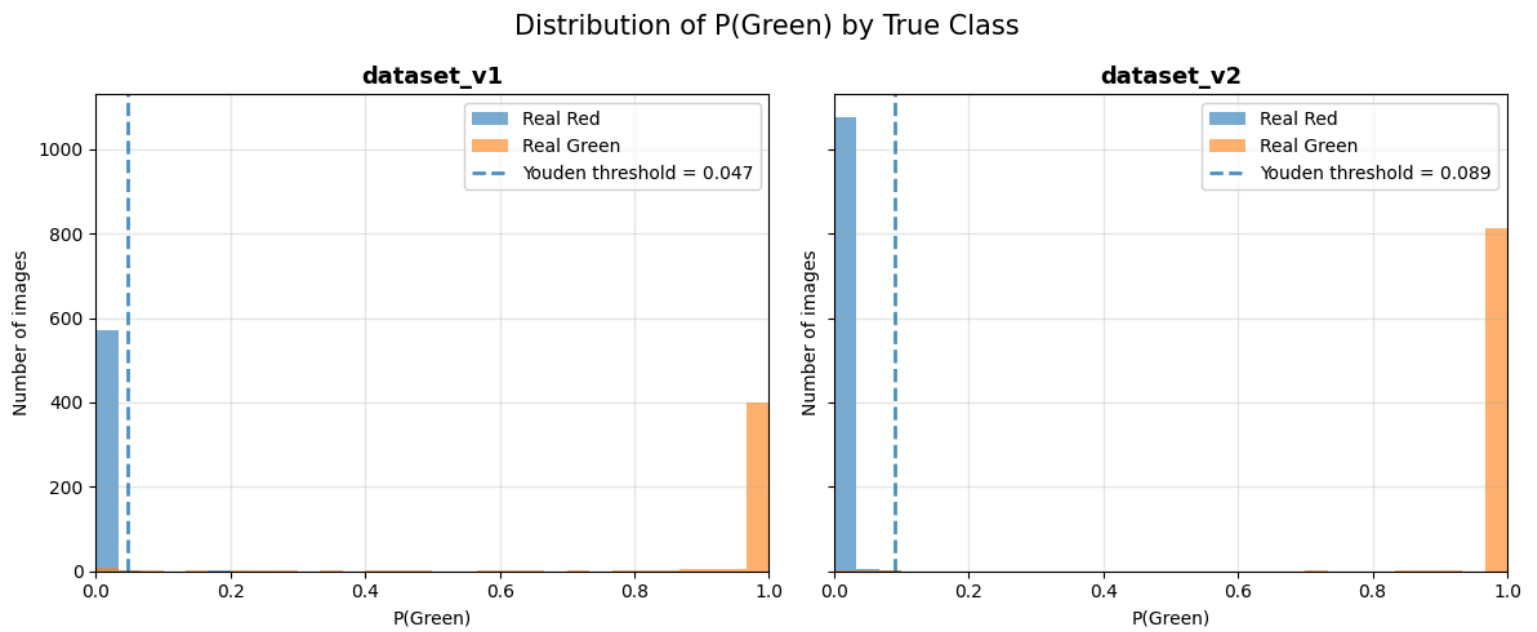


Figura 21. Distribución de P(Green) según clase y umbrales Youden para cada dataset.

Fuente: elaboración propia. [\(Volver\)](#)

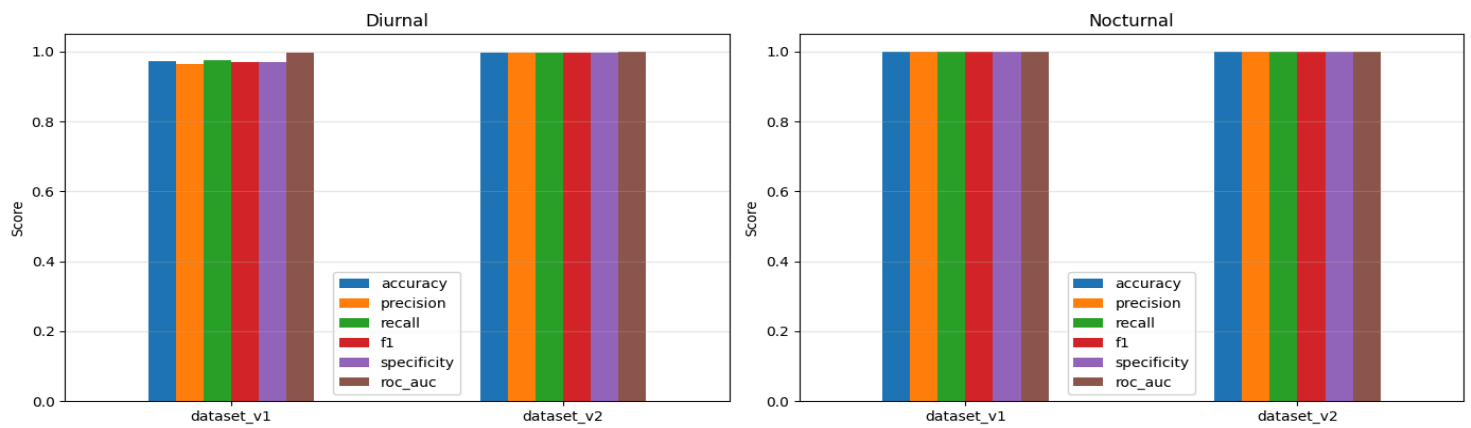


Figura 22. Resultados CNN según versión del dataset utilizando Youden Threshold.

Fuente: elaboración propia. [\(Volver\)](#)

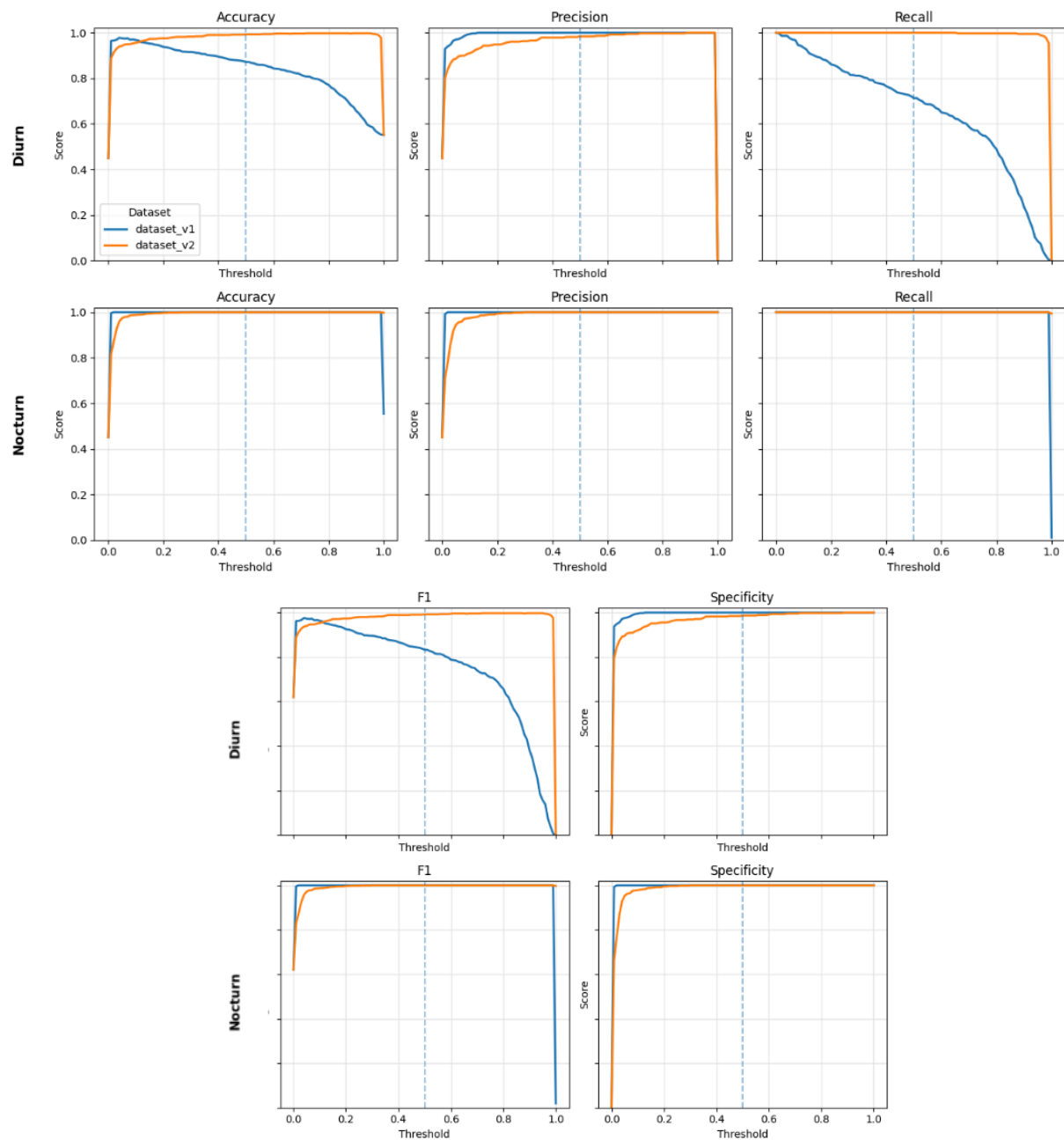


Figura 23. Evolución de métricas en CNN según versión del dataset y threshold sobre test.

Fuente: elaboración propia. [\(Volver\)](#)

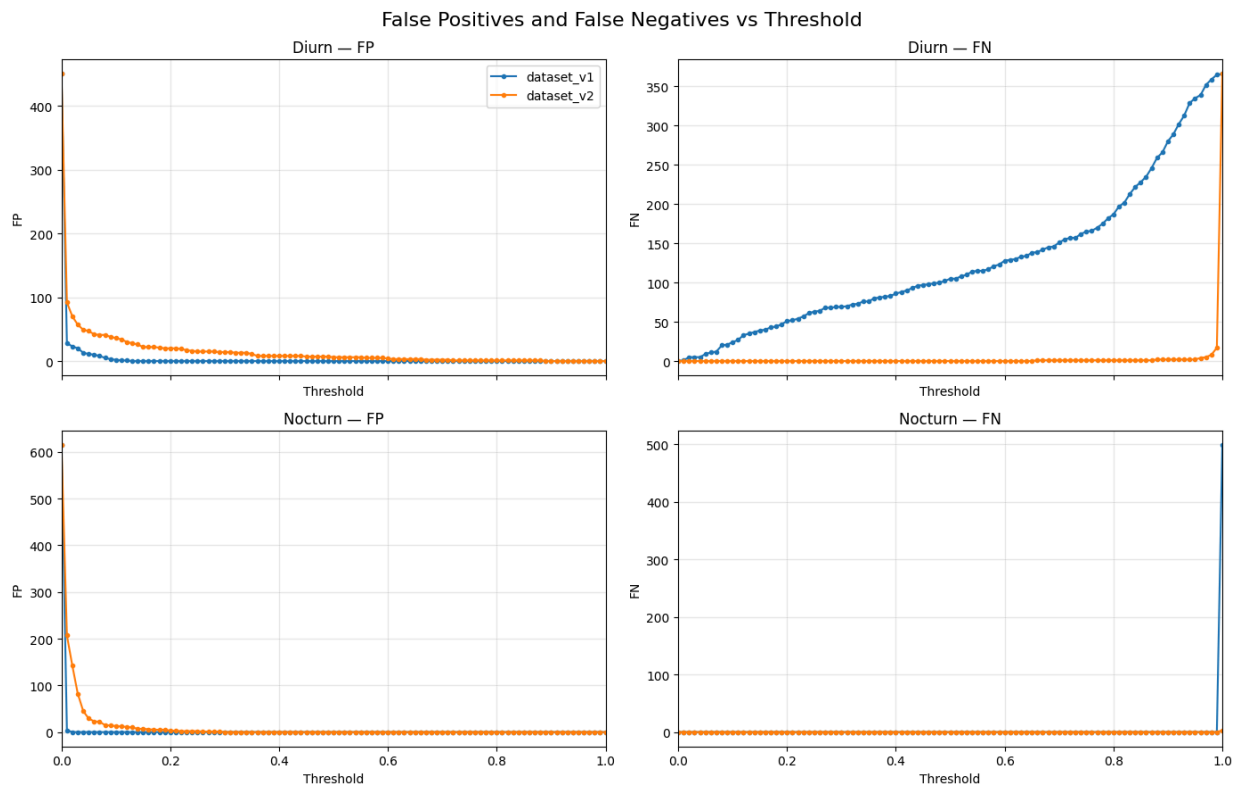


Figura 24. Evolución de falsos negativos y falsos positivos según threshold sobre test.

Fuente: elaboración propia. [\(Volver\)](#)

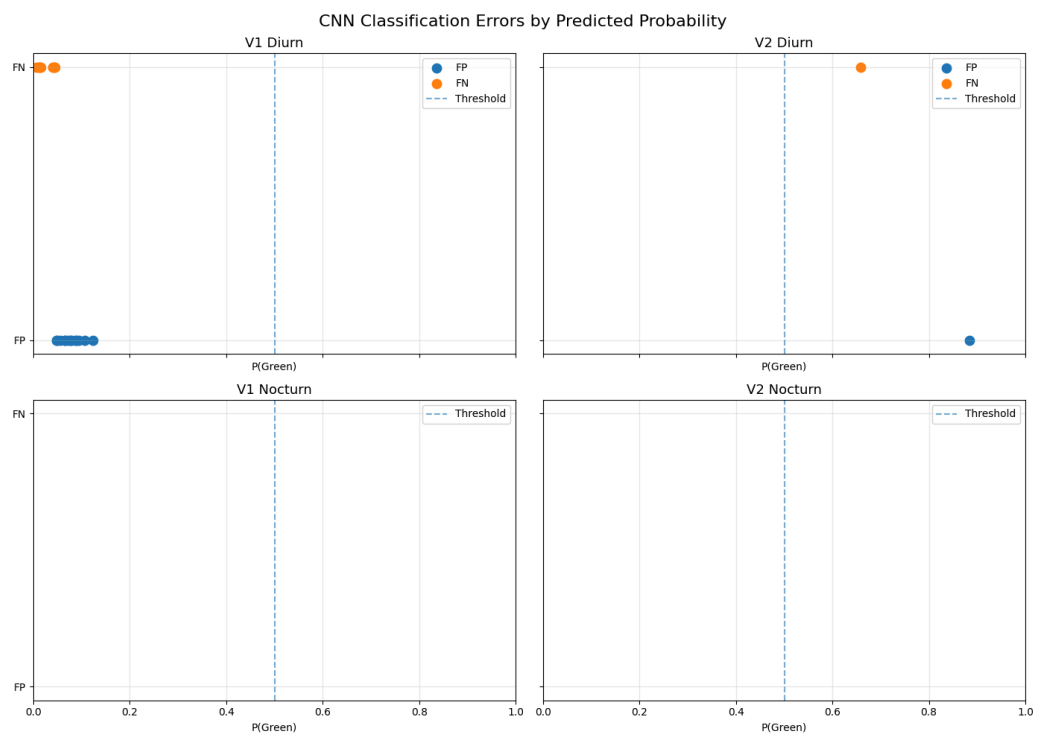


Figura 25. Falsos positivos y falsos negativos y su valor P(Green) para ambas versiones de CNN.

Fuente: elaboración propia. [\(Volver\)](#)

Distribution of P(Green) by True Class

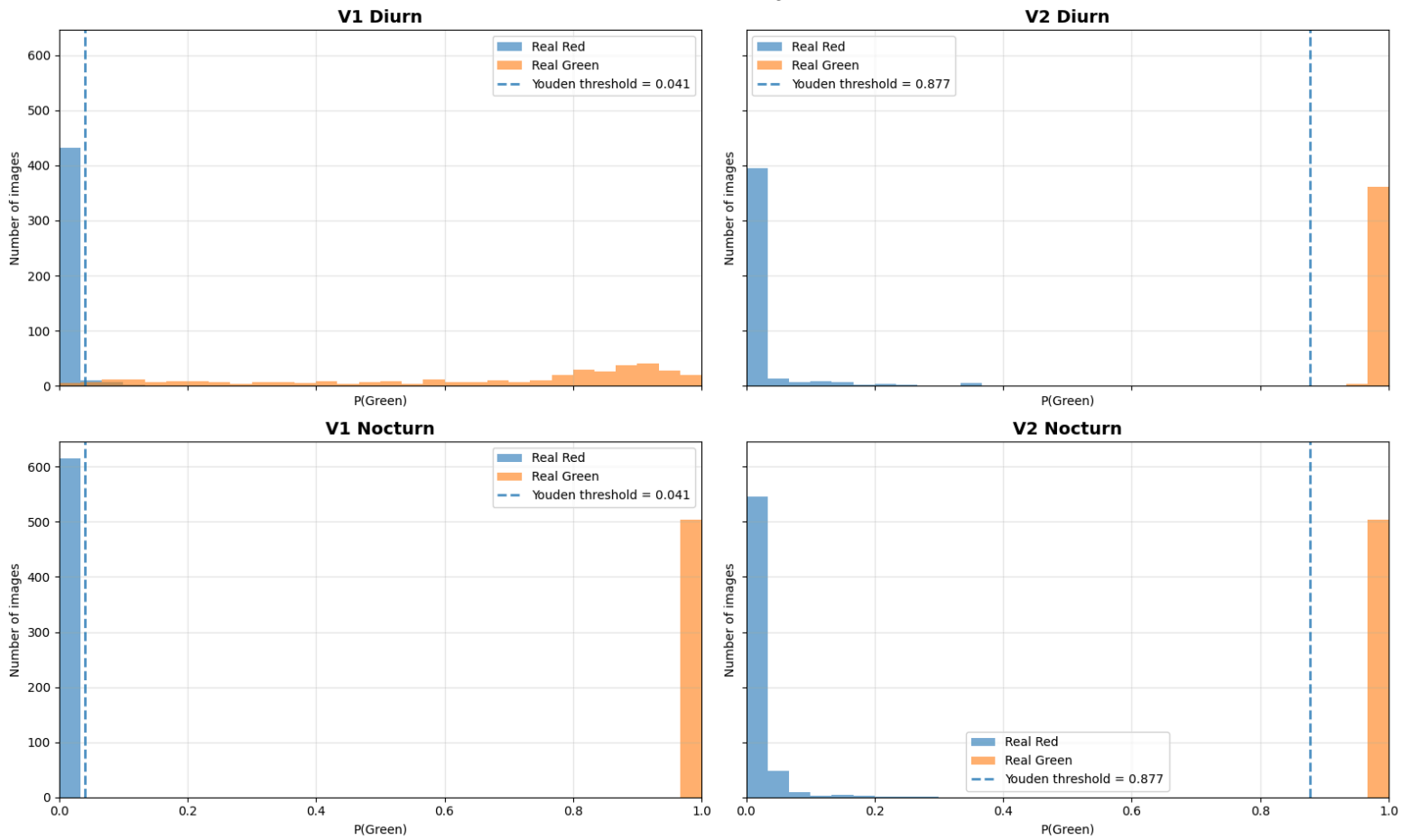


Figura 26. Distribución de $P(\text{Green})$ según clase y umbrales Youden para test.

Fuente: elaboración propia. [\(Volver\)](#)

FN | True: Green | Pred: Red
 $P(\text{Green})$: 0.658
 000117_frame000002_mS1Vk6udNA.jpg



FP | True: Red | Pred: Green
 $P(\text{Green})$: 0.884
 000130_frame000009_PTYNivELOc.jpg



Figura 27. Errores en test para la CNN dataset_v2.

Fuente: elaboración propia. [\(Volver\)](#)

Grad-CAM++ — CNN V2 Classification Errors — Diurn



Figura 28. Errores en test para la CNN dataset_v2 y Grad-CAM++.

Fuente: elaboración propia. [\(Volver\)](#)



Figura 29. Errores en test para la CNN dataset_v2 y Grad-CAM++.

Fuente: elaboración propia. [\(Volver\)](#)

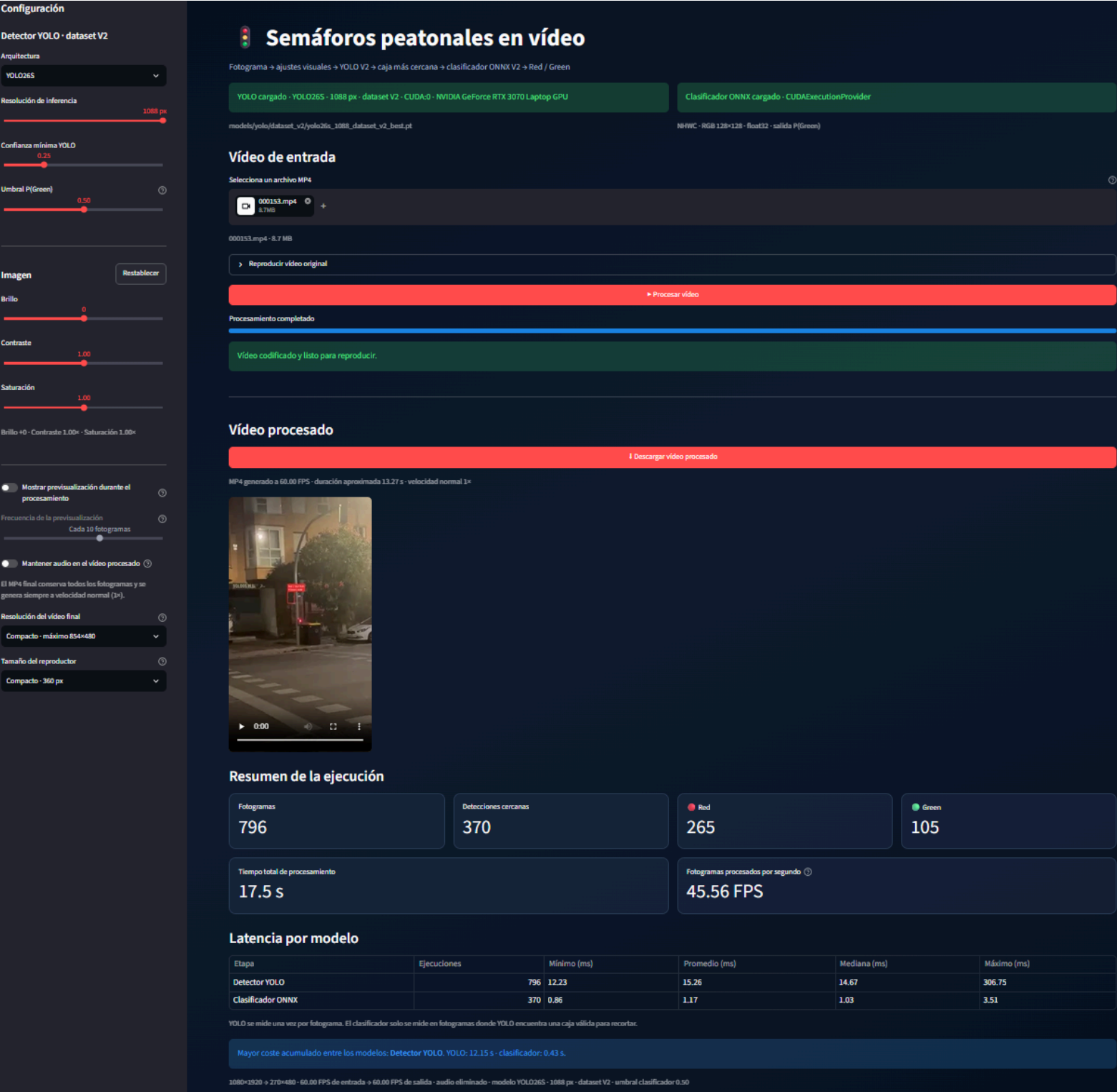




Figura 31. Semáforo nocturno apagado clasificado como Red.

Fuente: elaboración propia. [\(Volver\)](#)

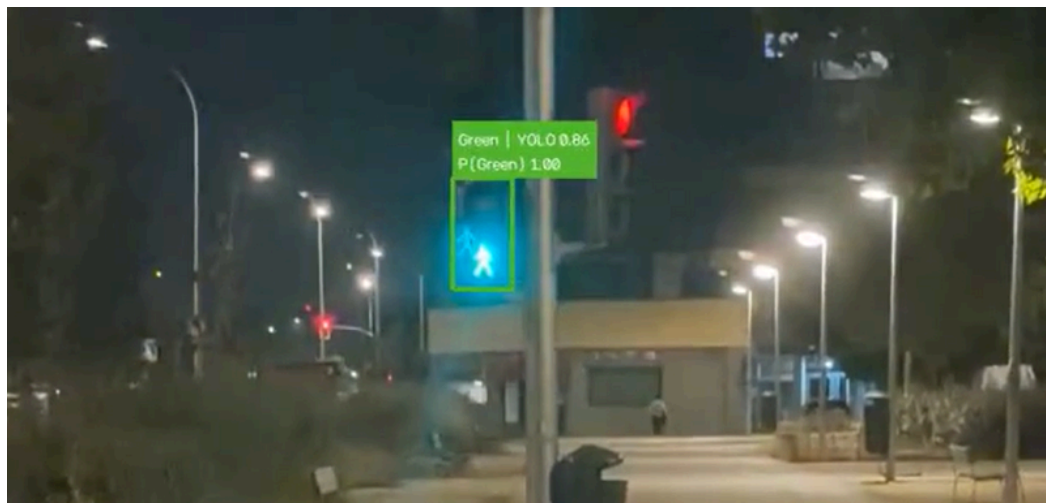


Figura 32. Semáforo nocturno correctamente clasificado como Green.

Fuente: elaboración propia. [\(Volver\)](#)

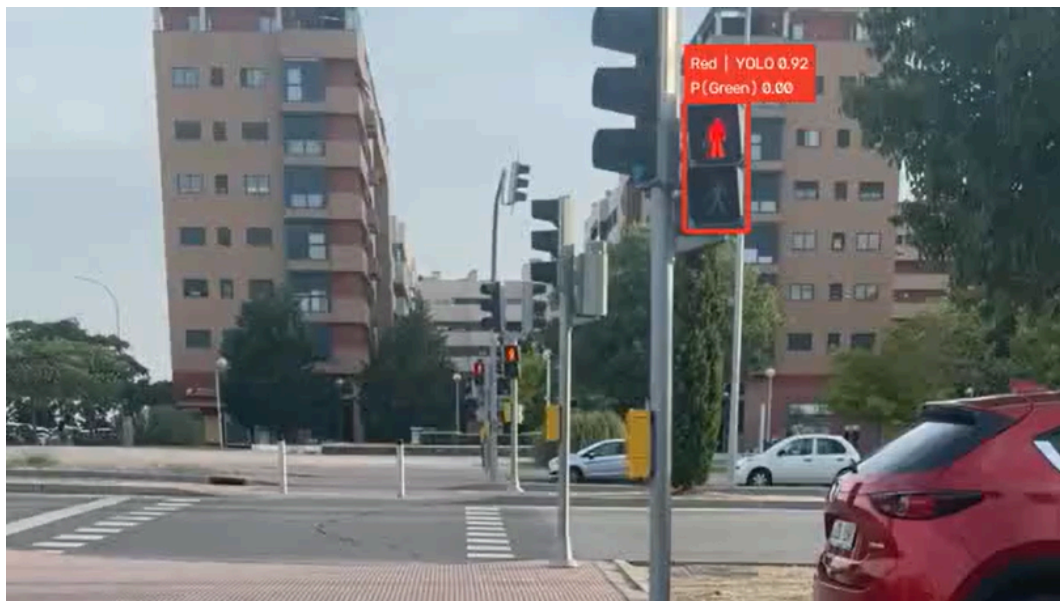


Figura 33. Detección y clasificación de un cruce regulado por múltiples semáforos.

Fuente: elaboración propia. [\(Volver\)](#)

VIII.II. TABLAS

Conjunto	Vídeos	Frames	Objetos	Frames/vídeo
Dataset v1	32	2197	3392	68.66
Dataset v2	99	4090	6332	41.31
Test diurno	39	718	816	18.41
Test nocturno	30	1119	1119	37.30

Tabla 1. Resumen de los conjuntos de datos obtenidos

Fuente: elaboración propia. [\(Volver\)](#)

Modelo	mAP ^{val} 50-95	params (M)
YOLO26n	40.9	2.4
YOLO26s	48.6	9.5
YOLO26m	53.1	20.4
YOLO26l	55.0	24.8
YOLO26x	57.5	55.7

Tabla 2. mAP50-95 y número de parámetros (en millones) según tipo de modelo YOLO. 640 x 640 píxeles de referencia.

Fuente: (Ultralytics, 2026.-a). [\(Volver\)](#)

Métrica		Dataset v1	Dataset v2	Test diurno	Test nocturno
Anchura (%)	Mediana	2.69	2.76	4.33	3.66
	Mínima	0.33	0.74	0.82	1.30
	Máxima	19.12	19.12	14.39	13.24
Altura (%)	Mediana	2.75	2.79	4.65	3.82
	Mínima	0.03	0.83	1.01	1.49
	Máxima	16.57	16.57	16.86	12.81
Área (%)	Mediana	7.44	7.67	20.28	14.13
	Mínima	0.01	0.67	0.83	2.07
	Máxima	256	256	237.85	164.1

Tabla 3. Estadísticas básicas de las bounding boxes según dataset.

Fuente: elaboración propia. [\(Volver\)](#)

Modelo	Resoluciones	Batch	Épocas	Paciencia	Optimizador
YOLO26n v1	640, 800, 960, 1088	8	75	20	auto
YOLO26s v1	640, 800, 960, 1088	8	75	20	auto
YOLO26n v2	640, 800, 960, 1088	8	75	20	auto
YOLO26s v2	640, 800	16	75	20	auto
YOLO26s v2	960, 1088	8	75	20	auto

Tabla 4. Hiperparámetros básicos del entrenamiento YOLO.

Fuente: elaboración propia. [\(Volver\)](#)

Test diurno		Precisión		Recall		mAP@50		mAP@50:95	
Modelo	Resolución	V1	V2	V1	V2	V1	V2	V1	V2
YOLO26n	640	0.8704	0.9304	0.7769	0.9019	0.8590	0.9598	0.5864	0.6672
YOLO26s	640	0.9117	0.9743	0.8866	0.9294	0.9373	0.9608	0.6354	0.7316
YOLO26n	800	0.9268	0.9759	0.8223	0.9399	0.9178	0.9715	0.6169	0.7537
YOLO26s	800	0.9266	0.9845	0.9138	0.9362	0.9496	0.9728	0.7286	0.7770
YOLO26n	960	0.9116	0.9819	0.8477	0.9313	0.9370	0.9705	0.6840	0.7680
YOLO26s	960	0.9489	0.9790	0.9558	0.9583	0.9873	0.9911	0.7617	0.8095
YOLO26n	1088	0.9230	0.9478	0.8965	0.9424	0.9538	0.9788	0.6553	0.7427
YOLO26s	1088	0.9606	0.9801	0.9607	0.9703	0.9899	0.9928	0.7444	0.8133

Tabla 5. Resultados del detector YOLO según dataset, modelo y resolución en test diurno.

Fuente: elaboración propia. [\(Volver\)](#)

Test nocturno		Precisión		Recall		mAP@50		mAP@50:95	
Modelo	Resolución	V1	V2	V1	V2	V1	V2	V1	V2
YOLO26n	640	0.8647	0.8319	0.8355	0.9072	0.8857	0.9241	0.4122	0.4786
YOLO26s	640	0.8979	0.9544	0.8212	0.8617	0.8936	0.9323	0.4655	0.5841
YOLO26n	800	0.8859	0.9215	0.8060	0.8820	0.8810	0.9203	0.4407	0.5638
YOLO26s	800	0.9099	0.9496	0.8755	0.8936	0.9271	0.9366	0.5924	0.6641
YOLO26n	960	0.9008	0.9262	0.8489	0.8990	0.9082	0.9431	0.5097	0.6019
YOLO26s	960	0.9454	0.9457	0.8834	0.9222	0.9350	0.9640	0.6048	0.6787
YOLO26n	1088	0.8967	0.9337	0.8409	0.8819	0.8997	0.9488	0.5045	0.6510
YOLO26s	1088	0.9133	0.9678	0.9240	0.8954	0.9391	0.9550	0.6051	0.6760

Tabla 6. Resultados del detector YOLO según dataset, modelo y resolución en test nocturno.

Fuente: elaboración propia. [\(Volver\)](#)

Resolución diurno	YOLO26n	YOLO26s	Diff. (%)
640	0.6672	0.7316	9.65
800	0.7537	0.7770	3.09
960	0.7680	0.8095	5.40
1088	0.7427	0.8133	9.51
		Promedio:	6.91

Tabla 7. mAP@50:95 en test diurno según modelo y resolución y diferencia en porcentaje.

Fuente: elaboración propia. [\(Volver\)](#)

Resolución nocturno	YOLO26n	YOLO26s	Diff. (%)
640	0.4786	0.5841	22.04
800	0.5638	0.6641	17.79
960	0.6019	0.6787	12.76
1088	0.6510	0.6760	3.84
		Promedio:	14.11

Tabla 8. mAP@50:95 en test nocturno según modelo y resolución y diferencia en porcentaje.

Fuente: elaboración propia. [\(Volver\)](#)

Conjunto	Imágenes	Red (%)	Green (%)
Dataset v1	1022	55.28	43.64
Dataset v2	1905	55.80	43.15
Test diurno	816	53.55	44.85
Test nocturno	1119	47.90	45.04

Tabla 9. Número de imágenes y distribución por clase de los datasets empleados en la CNN

Fuente: elaboración propia. [\(Volver\)](#)

Métrica	Dataset_v1	Dataset_v2
Accuracy	0.9911	0.9989
Precisión	0.9977	1.0000
Recall	0.9821	0.9975
F1-score	0.9898	0.9987
Specificity	0.9982	1.0000
False Positive	1	0
False Negative	8	2
ROC AUC	0.9979	0.9999
Youden Threshold utilizado	0.0465	0.0890

Tabla 10. Resultados del clasificador utilizando Youden según versión del dataset.

Fuente: elaboración propia. ([Volver](#))

Métrica	Dataset_v2 Youden	Dataset_v2 0.80
Accuracy	0.9989	0.9979
Precisión	1.0000	1.0000
Recall	0.9975	0.9951
F1-score	0.9987	0.9975
Specificity	1.0000	1.0000
False Positive	0	0
False Negative	2	4
ROC AUC	0.9999	0.9999
Threshold utilizado	0.0890	0.8000

Tabla 11. Comparativa de resultados del clasificador para dataset_v2 según threshold.

Fuente: elaboración propia. ([Volver](#))

	Dataset_v1		Dataset_v2	
Métrica	Diurno	Nocturno	Diurno	Nocturno
Accuracy	0.9730	1.0000	0.9975	1.0000
Precisión	0.9648	1.0000	0.9972	1.0000
Recall	0.9754	1.0000	0.9972	1.0000
F1-score	0.9701	1.0000	0.9972	1.0000
Specificity	0.9711	1.0000	0.9977	1.0000
False Positive	13	0	1	0
False Negative	9	0	1	0
ROC AUC	0.9981	1.0000	0.9999	1.0000
Threshold utilizado	0.047		0.8	

Tabla 12. Resultados de los clasificadores utilizando el conjunto de test.

Fuente: elaboración propia. [\(Volver\)](#)

VIII.III. DECLARACIÓN DE ORIGINALIDAD

DECLARACIÓN DE ORIGINALIDAD

Yo, Javier Galdeano Valenzuela, declaro que el TFM: “SISTEMA DE VISIÓN POR COMPUTADOR PARA LA DETECCIÓN Y CLASIFICACIÓN DE SEMÁFOROS PEATONALES COMO HERRAMIENTA DE ASISTENCIA PARA PERSONAS CON DISCAPACIDAD VISUAL” es totalmente original mío, que no ha sido presentado en ninguna otra universidad como TFM y que todas las fuentes que han sido utilizadas han sido adecuadamente citadas y aparecen en las referencias bibliográficas.

En Madrid, a 17 de Septiembre de 2026.

Firma: Javier Galdeano Valenzuela.

VIII.IV. USO RESPONSABLE DE IA GENERATIVA

DECLARACIÓN DE USO RESPONSABLE DE IA GENERATIVA

Yo, Javier Galdeano Valenzuela, declaro que durante el desarrollo del presente Trabajo de Fin de Máster se han utilizado herramientas de Inteligencia Artificial Generativa como apoyo en determinadas tareas del proyecto.

Su utilización se ha limitado a labores de asistencia en la implementación, revisión y mejora de código, así como en la generación o elaboración de determinados recursos gráficos y, cuando corresponda en la menor medida posible, tareas de apoyo a la redacción y revisión del texto.

La concepción del proyecto, su arquitectura, las decisiones técnicas y metodológicas, la adquisición y preparación de los datos, el diseño de los experimentos, la selección y configuración de los modelos, los criterios de evaluación, la validación del código y la interpretación de los resultados han sido realizadas por el autor.

Todo contenido generado o propuesto mediante herramientas de Inteligencia Artificial Generativa ha sido revisado, comprendido, adaptado y validado antes de su incorporación al trabajo. En ningún caso se han utilizado estas herramientas como sustitución del criterio técnico, la toma de decisiones o la responsabilidad académica del autor.

En Madrid, a 17 de septiembre de 2026.

Firma: Javier Galdeano Valenzuela.

VIII.V. DISPONIBILIDAD DEL REPOSITORIO

DECLARACIÓN DE DISPONIBILIDAD DEL REPOSITORIO

Yo, Javier Galdeano Valenzuela, declaro que el código desarrollado durante la realización del presente Trabajo de Fin de Máster, así como todo lo relevante para su corrección y réplica, se encuentra disponible en el siguiente repositorio de GitHub.

[Javier Galdeano Valenzuela - Pedestrian Traffic Light](#)

Debido al tamaño de determinados archivos, los vídeos originales junto con los recortes y algunos resultados no se encuentran almacenados directamente en el repositorio. No obstante, mediante los scripts y los notebooks se puede reconstruir el flujo de trabajo siempre que se disponga de los datos originales.

En Madrid, a 17 de septiembre de 2026.

Firma: Javier Galdeano Valenzuela.

